Execution Environments

Step by step: Windows, AD and SQL - Network (Cisco and Palo Alto) - and one EE that has it all

Build them, check them, add them to AAP | AAP 2.7, ansible-builder 3 | October 2026

What this is. A step-by-step guide to building three execution environments (EEs) for Ansible Automation Platform 2.7: one for **Windows, Active Directory and SQL** (SQL Server, MySQL, MariaDB), one for the **network** (Cisco and Palo Alto), and an **all-in-one** EE that has everything. Each one is built on top of **your base EE,** `base-ee:1.0`, and adds to everything it has. You build each with `ansible-builder` and podman, check it with one command, and add it to AAP. Every step says what to run and what you should see.

How to read it
Chapter 1 explains the three EEs. Do chapter 2 **once**. Then do the chapter of each EE you want (3, 4 or 5), and chapter 6 for each one you built. Chapter 8 lists what to do when something goes wrong. The build files are in the appendix, and in the repository's `ee/` folder. Rendered from `docs/EXECUTION_ENVIRONMENT.md` and `ee/` in the site-automation repository (version 0.81.0).

Host names and addresses in this guide are placeholders. Copy the files from the repository's ee/ folder, not from the PDF.

Contents

1. Start here 4

How the guide works . . . 4

Where each piece comes from . . . 5

2. Once, before any EE 6

Step 1. The build machine . . . 6

Step 2. Put the build files on the machine . . . 6

Step 3. Your base EE: base-ee:1.0 . . . 7

Step 4. Point the builds at your automation hub . . . 8

Step 5. Your hub token . . . 11

Step 6. Your CA certificates . . . 11

Step 7. Check that the build can reach PyPI . . . 11

3. EE 1: Windows, Active Directory and SQL 13

What is in it . . . 13

Step 1. Go to the folder . . . 13

Step 2. Build it . . . 14

Step 3. Check it . . . 14

Step 4. Check Kerberos to Active Directory . . . 14

Step 5. Put it into AAP . . . 14

Step 6. A first job . . . 14

Active Directory as an AAP inventory . . . 17

4. EE 2: Network - Cisco and Palo Alto 19

What is in it . . . 19

Step 1. Go to the folder . . . 19

Step 2. Build it . . . 19

Step 3. Check it . . . 20

Step 4. Put it into AAP . . . 20

Step 5. Cisco: a first job . . . 20

Step 6. Palo Alto: a first job . . . 21

5. EE 3: All-in-one 23

What is in it . . . 23

Step 1. Go to the folder . . . 23

Step 2. Build it . . . 24
Step 3. Check it . . . 24
Step 4. Put it into AAP . . . 24
Step 5. A first job . . . 24

6. Put an EE into AAP **25**

Step 1. Put the image where AAP can pull it . . . 25
Step 2. Add it in AAP . . . 25
Step 3. Use it . . . 25

7. Keep them up to date **27**

Pin the versions after the first good build . . . 27
0.33.0: git for the config backup . . . 27
0.33.1: a start script every user can read . . . 27
0.60.0: pymssql on a FIPS host . . . 27
Rebuild with a new tag . . . 28
Why three collections are pinned . . . 28
base-ee:1.0 already has some of these collections . . . 28
A new base EE . . . 29

8. When something goes wrong **30**

9. Appendix: the build files **33**

EE 1: Windows, AD and SQL - folder windows/ . . . 33
EE 2: Network (Cisco and Palo Alto) - folder network/ . . . 38
EE 3: All-in-one - folder all-in-one/ . . . 42
The same in every folder . . . 48
The scripts - folder ee/ . . . 51

1. Start here

An **execution environment (EE)** is the container image an AAP job runs in. It holds Ansible, the collections (the modules) and the Python libraries those modules need. A job can only use what its EE has. If a module is missing, the job fails with `couldn't resolve module/action`.

This guide builds three EEs, step by step. Each one starts from **your base EE,** `base-ee:1.0`, the one you built with the earlier EE guide. It keeps everything `base-ee:1.0` has and adds more. None of them is built on Red Hat's bare-minimum image.

EE	Image name	What it is for
Windows, AD and SQL	`site-ee-windows`	Windows servers over WinRM (Kerberos), Active Directory (and AD as an AAP inventory), SQL Server, MySQL and MariaDB
Network	`site-ee-network`	Cisco (IOS / IOS-XE, Nexus, ASA) and Palo Alto (PAN-OS firewalls and Panorama)
All-in-one	`site-ee-all`	Everything above, plus Linux, VMware, NetApp, ServiceNow and every site-automation job

Which ones do I build?
You can build all three, or only the ones you need. The all-in-one EE alone is enough for every job. The other two are smaller and focused: a Windows team or a network team gets only what it uses. The steps are the same for each EE.

How the guide works

- **Chapter 2** you do **once**: the build machine, the logins, the files. It covers all three EEs.
- **Chapters 3, 4 and 5** are one EE each. Do only the ones you want, in any order.
- **Chapter 6** puts an EE into AAP. Do it for each EE you build.
- Every step has a command to run and a **You should see** box. If you see something else, stop and look it up in chapter 8, *When something goes wrong*.
- Text like `aap.example.mil` or `EXAMPLE.MIL` is a placeholder. Use your own names.

Where each piece comes from

Piece	Comes from	Supported by
Your base EE, `base-ee:1.0`: RHEL 9, Python, ansible-core, ansible-runner and what you added to it	your build, on top of Red Hat's AAP image from `registry.redhat.io`	Red Hat, for the Red Hat parts
System packages (`bindep.txt`: `kinit`, `dig`, `ssh` ...)	Red Hat's RHEL repositories, through the build machine's subscription	Red Hat
Certified collections (`ansible.windows`, `microsoft.ad`, `cisco.ios` ...)	Red Hat Automation Hub, or your Private Automation Hub	Red Hat (partner collections: the vendor)
Community collections (`community.general`, `lowlydba.sqlserver` ...)	Ansible Galaxy, or your hub's *community* repository	the community
Python libraries (`pywinrm`, `pymssql`, `pan-os-python` ...)	**PyPI** (pypi.org), or your PyPI mirror	their authors, **not** Red Hat

The Python libraries are not RPMs. Red Hat does not sign or patch them, and a RHEL update does not change them. They change only when you rebuild the EE.

Tested, and what was not

All three EEs were built from these exact files in October 2026 with ansible-builder 3.2. They were built on top of a stand-in for `base-ee:1.0`: an EE built on Red Hat's public UBI 9 minimal image, with ansible-core 2.16, Python 3.12, and older Windows, AD and network collections already in it. `ee-check` said `ALL YES` for each EE. The collections the base already had were kept, except `ansible.windows`: it went from 2.8 to 3.9 because `community.windows` needs it. An earlier build of the same EEs on ansible-core 2.20 passed too. Every example playbook in chapters 3 and 4 passed a syntax check, and its modules got as far as connecting. All 60 site-automation playbooks passed a syntax check in the all-in-one EE. **Not tested:** your real `base-ee:1.0` and your hub. The collections came from Galaxy, so `redhat.rhel_system_roles` was left out. Your first build is that test, and chapter 8 has what to do if something differs.

2. Once, before any EE

Do this chapter one time. Everything in it is used by all three EEs.

Step 1. The build machine

You need a **RHEL 9** machine with a RHEL subscription. The AAP server works, or any RHEL 9 server that can reach your automation hub. Log in as a normal user (not root).

```shell
sudo dnf install -y podman ansible-builder unzip
ansible-builder --version
podman --version
```

`ansible-builder` comes from the AAP repository. The machine needs the *Ansible Automation Platform* subscription for it. Without one: `pip3 install --user ansible-builder`.

You should see
`ansible-builder --version` prints `3.` something, and `podman --version` prints a version. Both commands are found.

Step 2. Put the build files on the machine

The build files are in the repository's `ee/` folder, one folder per EE - and in every release zip. Copy the release zip (or the `ee/` folder of your clone) to the build machine, then:

```shell
cd ~ && unzip -q site-automation-0.32.0.zip 'site-automation-0.32.0/ee/*'   # your version
mv site-automation-0.32.0/ee ~/ee && rmdir site-automation-0.32.0
cd ~/ee && ls
```

You should see
`all-in-one build-ee.sh network windows`

Each folder has the same kinds of files:

File	What it holds	Do you change it?
`execution-environment.yml`	the recipe: the base image, the lists below, the build steps	only the base image's name, if yours differs (step 3)
`requirements.yml`	the collections	only to add or remove one
`requirements.txt`	the Python libraries (from PyPI) that no collection lists	only to add one
`bindep.txt`	the RHEL packages that no collection lists	only to add one
`ansible.cfg.example`	where the collections come from: your hub	you copy it to `ansible.cfg` (step 4)

File	What it holds	Do you change it?
`ca/`	your CA certificates	you put them in (step 5)
`check-ee.sh`, `check-list.txt`	`ee-check`: a test that runs inside the finished image	no
`krb5-ad-enctypes.conf`	Kerberos to Active Directory from a FIPS host (Windows and all-in-one only)	no
`freetds.tar.gz`	not in the zip: FreeTDS's source, which pymssql (SQL Server) is built with - step 7 gets it (Windows and all-in-one only)	no

Step 3. Your base EE: base-ee:1.0

Every EE in this guide starts from `base-ee:1.0`, the EE you built with the earlier EE guide. The new EEs keep everything it has and add to it. The build machine needs that image:

shell

```
podman images | grep base-ee
```

You should see
a line `localhost/base-ee 1.0 ...`.

Not there? Get it from wherever you keep it:

shell

```
podman login aap.example.mil
podman pull aap.example.mil/base-ee:1.0       # from your hub's registry
podman load -i base-ee-1.0.tar                # or from a file you saved
```

If you pulled it from the registry, give it the local name too. Then everything in this guide works as written, with no file to change:

shell

```
podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0
```

See what it has. These three commands show its ansible-core version, the Python that Ansible runs on, and its collections:

shell

```
podman run --rm localhost/base-ee:1.0 ansible --version
podman run --rm localhost/base-ee:1.0 bash -c 'head -1 $(command -v ansible)'
podman run --rm localhost/base-ee:1.0 ansible-galaxy collection list
```

> **You should see**
> `ansible [core 2.16...]` or `ansible [core 2.20...]`; both work with these files. Then `#!/usr/bin/python3.12`, and then the list of collections it already has.

`bash: /opt/builder/bin/entrypoint: Permission denied`**?** Every command in an EE starts through that script, and the user the EE runs as (1000) cannot read it: in this base EE it can be run but not read. To see it: `podman run --rm --entrypoint stat localhost/base-ee:1.0 -c '%a %U' /opt/builder/bin/entrypoint` (`711 root`, where `755 root` is right). Run the commands past it - `--entrypoint` and the command's first word before the image, the rest after it:

```shell
podman run --rm --entrypoint ansible localhost/base-ee:1.0 --version
podman run --rm --entrypoint bash localhost/base-ee:1.0 -c 'head -1 $(command -v ansible)'
podman run --rm --entrypoint ansible-galaxy localhost/base-ee:1.0 collection list
```

The EEs you build from it get a readable one: their `execution-environment.yml` sets it (0.33.1). The script, `build-ee.sh`, reads the base this way.

`WARN[0000] Failed to mount subscriptions ... permission denied` **above it?** podman, run as you, cannot read this machine's RHEL subscription files to hand them to the container. The commands above do not need them. The build may: it installs the system packages (`bindep.txt`) from the RHEL repositories. If the build stops at `No match for argument` or a repository error, build as root - the base EE into root's podman first, then the same build with `sudo`:

```shell
podman save -o base-ee-1.0.tar localhost/base-ee:1.0
sudo podman load -i base-ee-1.0.tar
sudo env INSECURE=1 bash build-ee.sh all-in-one 2     # or: sudo podman build ... by hand
```

The Python must match. The three `execution-environment.yml` files say `python_path: /usr/bin/python3.12`. If the second command shows another one, for example `#!/usr/bin/python3.11`, put that path in the three files:

```shell
cd ~/ee
sed -i 's|/usr/bin/python3.12|/usr/bin/python3.11|' */execution-environment.yml
```

No `base-ee:1.0`**?** Then use AAP's **Default** execution environment, Red Hat's `ee-supported-rhel9`, which already holds Red Hat's supported collections. Do not use the *Minimal* one, which is the bare minimum. Put this name in the three files instead:

```
registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
```

You need to log in to Red Hat's registry first, with your Red Hat account or a registry service account: `podman login registry.redhat.io`.

Step 4. Point the builds at your automation hub

The collections are downloaded from your automation hub during the build. Make `ansible.cfg` once, then copy it into each folder:

```shell
cd ~/ee
cp windows/ansible.cfg.example windows/ansible.cfg
vi windows/ansible.cfg
```

In `ansible.cfg`, change `aap.example.mil` to your hub's address in the three `url =` lines. AAP shows the right addresses: **Automation Content > Repositories >** a repository **> Copy CLI configuration**. Then copy the file to the other two folders:

```shell
cp windows/ansible.cfg network/ansible.cfg
cp windows/ansible.cfg all-in-one/ansible.cfg
```

> **You should see**
> `cat network/ansible.cfg` shows your hub's address, not `aap.example.mil`.

No Private Automation Hub? Use console.redhat.com for the certified collections and Galaxy for the community ones. Replace the whole `ansible.cfg` with:

```
[galaxy]
server_list = rh_certified, galaxy

[galaxy_server.rh_certified]
url = https://console.redhat.com/api/automation-hub/content/published/
auth_url = https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-connect/token

[galaxy_server.galaxy]
url = https://galaxy.ansible.com/
```

Keep the name `galaxy` for the Galaxy server. Galaxy needs no token, and it refuses one that is not its own (`HTTP Code: 401`).

Tell your hub admin that these collections must be in the hub. The certified ones are in the *rh-certified* repository. The ones below come from Galaxy, so the hub's *community* repository has to sync them:

- `community.general`, `community.windows`, `community.crypto`, `containers.podman`
- `lowlydba.sqlserver`
- `paloaltonetworks.panos` and `ansible.mysql`, if your *rh-certified* repository does not already have them

If a collection cannot be had, remove its line from `requirements.yml` and `check-list.txt`.

See what your hub lacks before you build. The build stops at the first collection it cannot find. `check-collections.sh` checks every one at once, as the build does: one your base EE already has in a version that fits is kept, the others are asked of your hub - with the base EE's own ansible-galaxy and the folder's `ansible.cfg` - and with `FETCH=1` it downloads the missing ones from Galaxy (no login needed) into `collections-to-upload/`, the newest version that fits, each with the ansible-core it needs:

```shell
cd ~/ee
FETCH=1 INSECURE=1 bash check-collections.sh all-in-one
```

> **You should see**
> a line per collection, `ok` with the version the base EE or your hub has, or `MISSING` and the file downloaded for it. Upload those (below), then run it again until it ends `all ... in the base EE or on your hub: build`.

And the versions for ansible-core 2.16. Three of them are pinned below their newest version (see *Why three collections are pinned*): `community.general` 11.x, `community.crypto` 2.x, `lowlydba.sqlserver` 2.x. A hub that synced only the newest has none of those, and the build stops with `Could not satisfy the following requirements: * community.general:<12.0.0`. Either the hub admin syncs them too - *Automation Content* > *Remotes* > *community* > *Edit*, in the YAML requirements a version range that takes them in (`version: ">=11.0.0"` for `community.general`, `">=2.0.0"` for the other two), then *Repositories* > *community* > *Sync* - or, when the build machine reaches the internet, those three come from Galaxy. In `requirements.yml`, a `source:` line under each of the three:

YAML

```
  - name: community.general             # community: hundreds of modules (ldap_*, mssql_*, nmcli,
    ...)
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
    source: https://galaxy.ansible.com
```

The same line under `community.crypto` and `lowlydba.sqlserver` - in one go:

shell

```
cd ~/ee/all-in-one
sed -i -e '/version: "<12.0.0"/a\    source: https://galaxy.ansible.com' \
       -e '/version: "<3.0.0"/a\    source: https://galaxy.ansible.com' requirements.yml
grep -c 'source: https://galaxy.ansible.com' requirements.yml     # 3
```

Every other collection still comes from your hub. (Galaxy as a fourth server in `ansible.cfg` would not do that: the build takes the newest version it finds anywhere, the certified ones from Galaxy too when Galaxy's is newer.)

Or upload the three into your hub (no remote to change, no Galaxy during the build). Download them - no login needed; `--no-check-certificate` behind TLS inspection - from Galaxy's *artifacts* address, the newest 11.x / 2.x of each:

shell

```
wget https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/comm
  unity-general-11.4.9.tar.gz
wget https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/comm
  unity-crypto-2.26.9.tar.gz
wget https://galaxy.ansible.com/api/v3/plugin/ansible/content/published/collections/artifacts/lowl
  ydba-sqlserver-2.8.1.tar.gz
```

Then in AAP: *Automation Content* > *Namespaces*, the namespace named as the file begins - `community`, `lowlydba` (*Create namespace* if it is not there) - *Upload collection*, each file; then *Automation Content* > *Collection Approvals*, *Approve*. They land in the *published* repository, which `ansible.cfg` already lists. A collection goes only into the namespace of its own name.

Step 5. Your hub token

The build needs a token to download from your hub. Get it in AAP: **Automation Content > API token > Load token**. For console.redhat.com: **console.redhat.com > Automation Hub > Connect to Hub > Load token**.

The token never goes in a file. Paste it into the shell when you build:

```shell
read -rs TOKEN
```

Paste the token, then press Enter. Nothing shows on the screen, and the token stays out of the shell history.

> **You should see**
> `echo ${#TOKEN}` prints a number bigger than 0: the token's length.

`TOKEN` lasts until you close the shell. In a new shell, run `read -rs TOKEN` again.

Step 6. Your CA certificates

Put your root and intermediate CA certificates into each folder's `ca/`, one `.pem` file each. A PEM file is text that starts with `-----BEGIN CERTIFICATE-----`. The build adds them to the EE's trust store, so TLS to LDAPS, vCenter, Panorama, ServiceNow and AAP is checked against your CA.

```shell
cd ~/ee
for d in windows network all-in-one; do cp ~/my-ca/*.pem $d/ca/; done
ls windows/ca
```

> **You should see**
> your `.pem` files, next to `README.txt`.

No CA files? The build still works. You just will not have your CA in the EE.

Step 7. Check that the build can reach PyPI

The Python libraries come from PyPI, pypi.org:

```shell
curl -sI https://pypi.org/simple/pip/ | head -1
```

> **You should see**
> `HTTP/2 200`, or `HTTP/1.1 200 OK`.

No answer? Your site needs a PyPI mirror, such as Nexus or Artifactory. Get its address from whoever runs it, for example `https://nexus.example.mil/repository/pypi/simple`. When you build, add this line to the `podman build` command, just before the line `-f context/Containerfile`:

```
  --build-arg PIP_INDEX_URL=https://nexus.example.mil/repository/pypi/simple \
```

Behind TLS inspection (the network replaces the sites' certificates with its own, so pip says `CERTIFICATE_VERIFY_FAILED`, ansible-galaxy `SSL: CERTIFICATE_VERIFY_FAILED`, podman `x509: certificate signed by unknown authority`): tell each one not to check, in the `podman build` command:

```
  --tls-verify=false \
  --build-arg PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org" \
  --build-arg ANSIBLE_GALAXY_CLI_COLLECTION_OPTS=--ignore-certs \
```

With a PyPI mirror, `PIP_TRUSTED_HOST` is its host instead (e.g. `nexus.example.mil`). Pulling or pushing an image: `podman pull --tls-verify=false ...`, `podman push --tls-verify=false ...`. The `ca/` folder (step 6) puts your CA in the finished EE for the jobs; it does not help the build itself, which runs before it.

And FreeTDS, for SQL Server (the Windows and all-in-one EEs). `pymssql`, which the `mssql_*` modules use, is built from its source during the build, with FreeTDS (SQL Server's network protocol) and the EE's own OpenSSL and Kerberos - PyPI's ready-made `pymssql` brings its own OpenSSL, which stops Python on a FIPS host. Put FreeTDS's source in each of the two folders:

shell

```
cd ~/ee/windows && curl -fo freetds.tar.gz
  https://www.freetds.org/files/stable/freetds-1.5.20.tar.gz
cp freetds.tar.gz ~/ee/all-in-one/
```

You should see
no output, and `ls -l freetds.tar.gz` about 5.5 MB. Behind TLS inspection, `curl -k`. No internet on the build machine: download that address on another machine and copy the file in. The build checks its SHA-256, so only that version works (`build-ee.sh` downloads it by itself).

You are ready. Go to the chapter of the EE you want.

Or let a script do it. `build-ee.sh` (in `~/ee`) does the checks of steps 3 to 7, then an EE's *Build it* and *Check it* steps, and chapter 6's step 1 when you ask. It stops at the first problem and says which step fixes it, and asks for your hub token (nothing shows as you paste it):

shell

```
cd ~/ee
bash build-ee.sh all-in-one 1                        # builds localhost/site-ee-all:1
PUSH=aap.example.mil bash build-ee.sh all-in-one 1   # ... and pushes it to your hub
SAVE=1 bash build-ee.sh all-in-one 1                 # ... or saves it as site-ee-all-1.tar
INSECURE=1 bash build-ee.sh all-in-one 1             # behind TLS inspection: no certificate
  checked
```

With a PyPI mirror (step 7), put `PIP_INDEX_URL=https://...` in front of the command. `INSECURE=1` (step 7, *Behind TLS inspection*) turns off every certificate check of the build: pip trusts PyPI's hosts (or your mirror's), ansible-galaxy `--ignore-certs`, podman `--tls-verify=false`; `PIP_TRUSTED_HOST="..."` alone turns off only pip's. The first argument is `windows`, `network` or `all-in-one`; the second is the version. At the end it prints what to enter in AAP (chapter 6, step 2).

You should see
`== ALL YES` (ee-check), and then the AAP fields: *Name*, *Image*, *Pull*.

3. EE 1: Windows, Active Directory and SQL

Image name: `site-ee-windows`. Folder: `~/ee/windows`. Built on: `base-ee:1.0`.

What is in it

Everything `base-ee:1.0` has, plus:

For	Collections	Python libraries and tools
Windows	`ansible.windows`, `community.windows`	`pywinrm` with Kerberos, `pypsrp`, `requests-kerberos`, `requests-ntlm`; `kinit`, `klist`
Active Directory	`microsoft.ad`	`pyspnego`, `gssapi`, `krb5`, `sansldap`, `ldap3`, `dnspython`; `dig`
SQL Server	`community.general` (`mssql_db`, `mssql_script`), `lowlydba.sqlserver`	`pymssql`, built in the build with FreeTDS and the EE's own OpenSSL and Kerberos (works on a FIPS host); `mssql-python`, Microsoft's driver, for your own Python
MySQL and MariaDB	`ansible.mysql`	`PyMySQL`
The report website on an SMB share		`smbprotocol` (Kerberos through `pyspnego` and `gssapi`)

There are two ways to reach SQL Server:

- **Over the network, port 1433, with a SQL login - or a domain account, with Kerberos.** These are the `community.general.mssql_*` modules. They run inside the EE.
- **On the Windows server, with Windows authentication.** These are the `lowlydba.sqlserver` modules: logins, databases, agent jobs, backups. They run on the SQL Server over WinRM and use the **dbatools** PowerShell module, which must be installed on that server.

Only want AD, or only SQL? Remove the other part's lines from `requirements.yml`, `requirements.txt` and `check-list.txt` before you build.

Step 1. Go to the folder

shell

```
cd ~/ee/windows
ls
```

You should see

`ansible.cfg ansible.cfg.example bindep.txt ca check-ee.sh check-list.txt execution-environment.yml freetds.tar.gz krb5-ad-enctypes.conf requirements.txt requirements.yml`

`ansible.cfg` is missing? Do chapter 2, step 4. `freetds.tar.gz` is missing? Chapter 2, step 7, *And FreeTDS*.

Step 2. Build it

```shell
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-windows:1 context
```

- The first command writes the build recipe into `context/`. The second one builds it.
- The first build takes **10 to 20 minutes**. It downloads everything and compiles the Kerberos libraries.
- `:1` is the version. Your next build is `:2`, and `:1` stays there to go back to.

> **You should see**
> at the end, `Successfully tagged localhost/site-ee-windows:1`.

Step 3. Check it

```shell
podman run --rm site-ee-windows:1 ee-check
```

`ee-check` lists every library, collection and tool this EE should have, with `YES` or `no`.

> **You should see**
> every line says `YES`, and the last line is `== ALL YES`.

Step 4. Check Kerberos to Active Directory

```shell
podman run --rm -it --network=host site-ee-windows:1 kinit svc_aap@EXAMPLE.MIL
```

Use a real domain account. The domain must be in **capital letters**.

> **You should see**
> `Password for svc_aap@EXAMPLE.MIL:`. Kerberos works. Press Ctrl+C, or type the password to get a ticket.

`Cannot find KDC for realm` or `KDC has no support for encryption type`: see chapter 8.

Step 5. Put it into AAP

Do chapter 6 with the image name `site-ee-windows:1` and the AAP name `Windows, AD and SQL EE`.

Step 6. A first job

Below are five small playbooks, one for each part of the EE. Put the ones you want in your project. Make a job template for each, with **this EE**, and run it.

1. Windows, `ee-test-windows.yml`. Inventory: your Windows servers. Credential: your Windows credential.

YAML

```
---
- name: Try the EE - Windows
  hosts: all
  gather_facts: false
  tasks:
    - name: WinRM works
      ansible.windows.win_ping:
```

You should see
each Windows server green, with `ping: pong`.

2. Active Directory, `ee-test-ad.yml`. It runs on a domain controller, or on a server with RSAT. On the template, set **Limit** to one domain controller, for example `dc01.example.mil`, and use the same Windows credential.

YAML

```
---
- name: Try the EE - Active Directory
  hosts: all
  gather_facts: false
  tasks:
    - name: The computer accounts in the Servers OU
      microsoft.ad.object_info:
        ldap_filter: (objectClass=computer)
        search_base: OU=Servers,DC=example,DC=mil
        properties: [dNSHostName, operatingSystem]
      register: ad
    - name: How many
      ansible.builtin.debug:
        msg: "{{ ad.objects | length }} computer accounts"
```

You should see
the number of computer accounts in that OU.

3. SQL Server with a SQL login, `ee-test-mssql.yml`. It runs inside the EE and connects to the server's port 1433.

YAML

```
---
- name: Try the EE - SQL Server with a SQL login
  hosts: localhost
  gather_facts: false
  tasks:
    - name: The databases and their state
      community.general.mssql_script:
        login_host: sql01.example.mil
        login_user: "{{ mssql_user }}"
        login_password: "{{ mssql_password }}"
        db: master
        script: SELECT name, state_desc FROM sys.databases
      register: dbs
    - name: Show them
      ansible.builtin.debug:
        var: dbs.query_results
```

For a first try, give `mssql_user` and `mssql_password` when you launch: on the template, tick **Prompt on launch** next to **Variables**. For real jobs, keep them in a credential. Chapter 4, step 6 shows how to make one.

> **You should see**
> the list of databases, each with `ONLINE`.

3b. SQL Server with a domain account (Kerberos), `ee-test-mssql-kerberos.yml`. The same, as a domain account: a Kerberos ticket first (`kinit`, in the EE), then no `login_user` - pymssql then logs in with the ticket.

YAML

```
---
- name: Try the EE - SQL Server with Kerberos
  hosts: localhost
  gather_facts: false
  tasks:
    - name: A Kerberos ticket for the domain account
      ansible.builtin.command: kinit {{ mssql_kerberos_user }}
      args:
        stdin: "{{ mssql_kerberos_password }}"
      changed_when: false
      no_log: true
    - name: The databases, as that account
      community.general.mssql_script:
        login_host: sql01.example.mil
        db: master
        script: SELECT SUSER_SNAME() AS login_name, name, state_desc FROM sys.databases
      register: dbs
    - name: Show them
      ansible.builtin.debug:
        var: dbs.query_results
```

`mssql_kerberos_user` is the account as `svc_aap@EXAMPLE.MIL` (the domain in capital letters). `login_host` is the server's **full DNS name**: the ticket asked for is `MSSQLSvc/sql01.example.mil:1433`, which must be the SQL Server service's SPN in AD (`setspn -L` on its service account) - an IP address or a short name gets no ticket. Run it without Check (in Check the `kinit` is skipped).

> **You should see**
> the databases, and in `login_name` the domain account (`EXAMPLE\svc_aap`).

3c. Your own Python with Microsoft's driver, `mssql-python`**.** The `mssql_*` modules use pymssql; for a script of your own, `mssql-python` (with Microsoft's ODBC Driver 18 inside it - nothing to install - using the EE's own OpenSSL and Kerberos) is in the EE too. With a ticket from `kinit`, as in *3b*:

```
import mssql_python
conn = mssql_python.connect("Server=sql01.example.mil,1433;Database=master;Trusted_Connection=yes;
  Encrypt=yes")
for row in conn.cursor().execute("SELECT SUSER_SNAME(), @@VERSION"):
    print(row)
```

`Encrypt=yes` checks the server's certificate against the EE's CAs (`ca/`, chapter 2, step 6); a SQL login instead of the ticket: `UID=...;PWD=...` in place of `Trusted_Connection=yes`. `No Kerberos credentials available`: no `kinit` in that job.

4. SQL Server with Windows authentication, `ee-test-sqlwin.yml`. It runs on the SQL Server itself, over WinRM. Put your SQL Servers in an inventory group, here `sqlservers`, and use your Windows credential.

YAML

```
---
- name: Try the EE - SQL Server with Windows authentication
  hosts: sqlservers
  gather_facts: false
  tasks:
    - name: The instance
      lowlydba.sqlserver.instance_info:
        sql_instance: localhost
      register: sql
    - name: Show it
      ansible.builtin.debug:
        var: sql.data
```

It needs **dbatools** on each SQL Server. Run this once, in PowerShell as an administrator: `Install-Module dbatools -Scope AllUsers`. If the servers have no internet, run `Save-Module dbatools -Path C:\temp` on a machine that does. Then copy both folders it makes into `C:\Program Files\WindowsPowerShell\Modules` on each SQL Server.

> **You should see**
> the instance's name, version and edition.

5. MySQL or MariaDB, `ee-test-mysql.yml`. It runs inside the EE and connects to the server's port 3306. Give `db_user` and `db_password` the same way as in playbook 3.

YAML

```
---
- name: Try the EE - MySQL or MariaDB
  hosts: localhost
  gather_facts: false
  tasks:
    - name: The databases
      ansible.mysql.mysql_info:
        login_host: db01.example.mil
        login_user: "{{ db_user }}"
        login_password: "{{ db_password }}"
        filter: [databases, version]
      register: db
    - name: Show the version
      ansible.builtin.debug:
        var: db.version
```

> **You should see**
> the server's version, for example `full: 10.11.9-MariaDB`.

Active Directory as an AAP inventory

This EE can also read the computer accounts from AD and turn each one into an AAP host. It uses the `microsoft.ad.ldap` inventory plugin, and nothing is installed on a domain controller.

1. Put a file in your project whose name ends in `microsoft.ad.ldap.yml`, for example `inventories/ad/servers.microsoft.ad.ldap.yml`:

YAML

```
plugin: microsoft.ad.ldap
server: dc01.example.mil          # leave it out and the DCs are found in DNS
tls_mode: ldaps                   # LDAP over TLS, port 636, checked against your CA
auth_protocol: simple             # on a FIPS host: simple over LDAPS, or kerberos - never ntlm
search_base: OU=Servers,DC=example,DC=mil
filter: (operatingSystem=*Server*)
attributes:
  operatingSystem:
    os_name:
keyed_groups:
  - key: os_name
    prefix: os
```

2. Make a credential type for the account. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**, name `AD LDAP bind`.

Input configuration

```
fields:
  - id: username
    type: string
    label: User (user@EXAMPLE.MIL)
  - id: password
    type: string
    label: Password
    secret: true
required: [username, password]
```

Injector configuration

```
env:
  MICROSOFT_AD_LDAP_USERNAME: "{{ username }}"
  MICROSOFT_AD_LDAP_PASSWORD: "{{ password }}"
```

3. Make a credential of that type. A normal, read-only domain account is enough.

4. In your inventory, go to **Sources > Create source**:

- **Source:** *Sourced from a Project*
- **Project:** your project
- **Inventory file:** the file from part 1
- **Credential:** `AD LDAP bind`
- **Execution environment:** `Windows, AD and SQL EE`

Then press **Sync**.

> **You should see**
> the sync goes green, and the hosts show up in groups such as `os_Windows_Server_2022_Standard`.

4. EE 2: Network - Cisco and Palo Alto

Image name: `site-ee-network`. Folder: `~/ee/network`. Built on: `base-ee:1.0`.

What is in it

Everything `base-ee:1.0` has, plus:

For	Collections	Python libraries and tools
Connections and helpers	`ansible.netcommon` (network_cli, netconf, httpapi), `ansible.utils`	`ansible-pylibssh`, `paramiko`, `ncclient`, `netaddr`, `textfsm`, `ttp`, `jmespath`; `ssh`, `sshpass`
Cisco	`cisco.ios` (IOS / IOS-XE), `cisco.nxos` (Nexus), `cisco.asa` (ASA)	(from the connections above)
Palo Alto	`paloaltonetworks.panos` (firewalls and Panorama)	`pan-os-python`, `pan-python`, `panos-upgrade-assurance`, `xmltodict`
The report website on an SMB share		`smbprotocol` (Kerberos through `pyspnego` and `gssapi`)
Config backups	-	`git` (the network config backup's repository)

Cisco is reached over SSH. Palo Alto is reached over HTTPS, through the PAN-OS XML API. Only need one vendor? Remove the other one's lines from `requirements.yml` and `check-list.txt` before you build.

Step 1. Go to the folder

shell

```
cd ~/ee/network
ls
```

You should see
`ansible.cfg ansible.cfg.example bindep.txt ca check-ee.sh check-list.txt execution-environment.yml requirements.txt requirements.yml`

Step 2. Build it

shell

```
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-network:1 context
```

This one is quicker, about 5 to 10 minutes. Nothing has to be compiled.

You should see
at the end, `Successfully tagged localhost/site-ee-network:1`.

Step 3. Check it

shell
```
podman run --rm site-ee-network:1 ee-check
```

You should see
every line says `YES`, and the last line is `== ALL YES`.

Step 4. Put it into AAP

Do chapter 6 with the image name `site-ee-network:1` and the AAP name `Network EE (Cisco and Palo Alto)`.

Step 5. Cisco: a first job

1. The inventory. Put your Cisco devices in a group, for example `ios`, and give the group these variables. In AAP: the inventory **> Groups > ios > Variables**.

YAML
```
ansible_connection: ansible.netcommon.network_cli
ansible_network_os: cisco.ios.ios
ansible_network_cli_ssh_type: libssh
ansible_become: true
ansible_become_method: enable
```

Nexus: `ansible_network_os: cisco.nxos.nxos`. ASA: `ansible_network_os: cisco.asa.asa`.

2. The credential. Use a *Machine* credential: the username, the password, and the enable password as the **Privilege Escalation Password**.

3. The playbook, `ee-test-cisco.yml`:

YAML
```
---
- name: Try the network EE - Cisco
  hosts: ios
  gather_facts: false
  tasks:
    - name: Facts
      cisco.ios.ios_facts:
        gather_subset: min
    - name: What it runs
      ansible.builtin.debug:
        msg: "{{ ansible_net_hostname }} runs {{ ansible_net_version }}"
```

You should see
each device green, with its name and IOS version.

Step 6. Palo Alto: a first job

The Palo Alto modules run inside the EE and talk to the firewall over HTTPS. They take the login from a `provider` setting, not from a Machine credential, so you make a small credential type for it.

1. The credential type. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**, name `Palo Alto PAN-OS`.

Input configuration

```
fields:
  - id: username
    type: string
    label: PAN-OS user
  - id: password
    type: string
    label: Password
    secret: true
required: [username, password]
```

Injector configuration

```
extra_vars:
  panos_username: "{{ username }}"
  panos_password: "{{ password }}"
```

Then make a credential of that type, with a PAN-OS admin account. A read-only admin is enough for this test.

2. The inventory. Put the firewalls (and Panorama) in a group, for example `panos`, with these group variables:

YAML

```
ansible_connection: local
ansible_python_interpreter: "{{ ansible_playbook_python }}"
```

3. The playbook, `ee-test-panos.yml`:

YAML

```
---
- name: Try the network EE - Palo Alto
  hosts: panos
  gather_facts: false
  tasks:
    - name: System facts
      paloaltonetworks.panos.panos_facts:
        provider:
          ip_address: "{{ ansible_host | default(inventory_hostname) }}"
          username: "{{ panos_username }}"
          password: "{{ panos_password }}"
        gather_subset: [system]
    - name: What it runs
      ansible.builtin.debug:
        msg: "{{ ansible_net_hostname }} runs PAN-OS {{ ansible_net_version }}"
```

On the job template, choose the `Palo Alto PAN-OS` credential, not a Machine credential.

You should see
each firewall green, with its name and PAN-OS version.

`certificate verify failed`: the firewall's HTTPS certificate is not from a CA in `ca/`. See chapter 8.

5. EE 3: All-in-one

Image name: `site-ee-all`. Folder: `~/ee/all-in-one`. Built on: `base-ee:1.0`.

What is in it

Everything `base-ee:1.0` has, everything in EE 1 and EE 2, plus:

For	Collections	Python libraries
Linux	`ansible.posix`, `ansible.utils`, `community.general`, `community.crypto`, `containers.podman`, `redhat.rhel_system_roles`	`jmespath`, `netaddr`, `dnspython`, `cryptography`
VMware	`vmware.vmware`	`pyVmomi`, `vmware-vcenter` (the vSphere Automation SDK)
NetApp	`netapp.ontap`	`requests`, `solidfire-sdk-python` and the others the collection lists
ServiceNow	`servicenow.itsm` (tickets, and the CMDB as an AAP inventory)	`requests`
Times in your time zone		`tzdata` (the system package; `report_time_zone`)

This is the EE for the site-automation jobs. `redhat.rhel_system_roles` is only on Red Hat's Automation Hub. If your hub does not have it, remove its line from `requirements.yml` and `check-list.txt`.

Built the all-in-one EE before?
If you built `site-ee-all:1` from the earlier EE guide, this version adds Palo Alto, `lowlydba.sqlserver`, `smbprotocol` (the report website on an SMB share) and `tzdata`. Build it as `site-ee-all:2` instead of `:1` in the commands below, so `:1` stays there to go back to.

Step 1. Go to the folder

shell

```
cd ~/ee/all-in-one
ls
```

You should see
`ansible.cfg ansible.cfg.example bindep.txt ca check-ee.sh check-list.txt execution-environment.yml freetds.tar.gz krb5-ad-enctypes.conf requirements.txt requirements.yml`

`freetds.tar.gz` is missing? Chapter 2, step 7, *And FreeTDS*.

Step 2. Build it

```shell
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-all:1 context
```

This is the biggest build, about **15 to 25 minutes**.

> **You should see**
> at the end, `Successfully tagged localhost/site-ee-all:1`.

Step 3. Check it

```shell
podman run --rm site-ee-all:1 ee-check
podman run --rm -it --network=host site-ee-all:1 kinit svc_aap@EXAMPLE.MIL
```

> **You should see**
> `== ALL YES` at the end of the first command. The second one shows `Password for svc_aap@EXAMPLE.MIL:`; press Ctrl+C.

Step 4. Put it into AAP

Do chapter 6 with the image name `site-ee-all:1` and the AAP name `All-in-one EE`.

Step 5. A first job

Every first job of EE 1 and EE 2 works with this EE too. For site-automation, run a template you know with the **All-in-one EE**, for example *Health check* in Check mode.

> **You should see**
> the job is green, just as it was with its old EE.

6. Put an EE into AAP

Do this chapter once for each EE you built. The examples use `site-ee-windows:1`. For the other EEs, change the name.

Step 1. Put the image where AAP can pull it

With a registry (a Private Automation Hub has one):

```shell
podman login aap.example.mil
podman tag site-ee-windows:1 aap.example.mil/site-ee-windows:1
podman push aap.example.mil/site-ee-windows:1
```

You should see
`Writing manifest to image destination`, and no error.

Without a registry: save the image as a file, and load it on **every AAP execution node**, as the user AAP runs jobs as:

```shell
podman save -o site-ee-windows-1.tar site-ee-windows:1    # on the build machine
podman load -i site-ee-windows-1.tar                      # on each execution node
```

You should see
`Loaded image: localhost/site-ee-windows:1` on each node.

Step 2. Add it in AAP

Go to **Automation Execution > Infrastructure > Execution Environments > Create execution environment**:

Field	With a registry	Loaded from a file
Name	`Windows, AD and SQL EE`	`Windows, AD and SQL EE`
Image	`aap.example.mil/site-ee-windows:1`	`localhost/site-ee-windows:1`
Pull	*Only pull the image if not present before running*	*Never pull container before running*
Registry credential	your hub's registry credential	none

Press **Create execution environment**.

Step 3. Use it

Open a job template, choose the EE under **Execution environment**, and save. You can also choose it on an inventory source, or set it as your organization's default.

Job templates for	Use
Windows servers, Active Directory, SQL Server, MySQL, MariaDB	`Windows, AD and SQL EE`
Cisco switches, routers, ASA; Palo Alto firewalls, Panorama	`Network EE (Cisco and Palo Alto)`
site-automation, Linux, VMware, NetApp, ServiceNow, or a mix	`All-in-one EE`

You should see
the job's **Details** tab shows the new EE, and the first job (chapters 3 to 5) is green.

Many templates at once: the site-automation job **AAP - set execution environment** (`playbooks/aap_ee.yml`, SETUP_AAP.md) puts one EE on every template of your site-automation project - or on those on a given EE now - in one run:

```yaml
aap_ee_name: "All-in-one EE"
aap_ee_project: "Site automation"           # your project's name in AAP
aap_ee_exclude: [Network config backup]     # jobs that stay on another EE
```

Run it with **Check** first: it lists every template it would change, from which EE to which.

A rebuild, published: push it with the next tag (chapter 7), then run the same job with the EE and its new image

- every template already on that EE runs the new image; none need be picked:

```yaml
aap_ee_name: "All-in-one EE"
aap_ee_image: aap.example.mil/site-ee-all:3     # the tag you just pushed
```

(Or change the EE's **Image** in AAP by hand.) Never push a rebuild under a tag already on the hub: an execution node that has `:1` keeps it - the job refuses the image the EE has now.

7. Keep them up to date

Pin the versions after the first good build

A new build downloads the newest collections, so two builds can differ. Once a build works, pin each collection to the version it got. `ee-check` shows the versions:

```shell
podman run --rm site-ee-windows:1 ee-check
```

Then write each version into `requirements.yml`, for example:

```YAML
  - name: microsoft.ad
    version: "==1.12.1"
```

0.33.0: git for the config backup

0.33.0 adds `git` to the network and all-in-one EEs: the network config backup (`net_config_store: git`) runs it on the AAP side, and they did not have it. (`smbprotocol`, for the report website on an SMB share, came in 0.32.0.) If you use the config backup with a git repository, build those two EEs again with the next tag (below), from the 0.33.0 files - keep your own `ansible.cfg` and `ca/` files - and check:

```shell
podman run --rm site-ee-all:3 ee-check | grep -iE 'smbprotocol|git'
#   YES  smbprotocol
#   YES  git
```

`git-core` is in the base image's own RHEL repositories: nothing else changes.

0.33.1: a start script every user can read

Every command in an EE starts through `/opt/builder/bin/entrypoint`, and an EE runs as user 1000. ansible-builder copies that script, `ee-check`'s list and the Kerberos settings in with the modes they have on the build machine and in the base image: a script user 1000 can run but not read answers `bash: /opt/builder/bin/entrypoint: Permission denied`. The three `execution-environment.yml` files of 0.33.1 make them readable, whatever modes they came with. An EE that shows that error: build it again from the 0.33.1 files with the next tag (below). One that runs `ee-check` is fine as it is.

0.60.0: pymssql on a FIPS host

PyPI's ready-made `pymssql` brings its own OpenSSL 1.1.1k: on a FIPS host it stops Python at `import pymssql` (`FATAL FIPS SELFTEST FAILURE`), so an EE built on a FIPS machine said `no pymssql`, and it was left out. From 0.60.0 the Windows and all-in-one EEs build pymssql from its source, with FreeTDS 1.5.20 (`freetds.tar.gz`, chapter 2, step 7) and the EE's own OpenSSL 3 and Kerberos - the SQL Server modules work on a FIPS host, with a SQL login or a domain account's Kerberos ticket (chapter 3, step 6, *3b*). Nothing of FreeTDS or the compiler stays in the image. Microsoft's own driver, `mssql-python`, is in both EEs too, for your own Python (*3c*). To get them:

1. Take 0.60.0's `execution-environment.yml` and `check-ee.sh` into your EE folders (keep your own `ansible.cfg`, `ca/`, and any lines you took out of the other files), and put `pymssql` back in `requirements.txt` and `lib pymssql pymssql` in `check-list.txt` if you took them out; add `mssql-python` and `lib mssql_python mssql-python`.
2. Put `freetds.tar.gz` in the folder (chapter 2, step 7), or let `build-ee.sh` download it.
3. Build with the next tag, check it - `ee-check` now also says `YES no library with its own OpenSSL 1.1 (FIPS)` - push it, and give the EE in AAP its new image (chapter 6, step 3, *A rebuild*).

shell

```
podman run --rm site-ee-all:3 ee-check | grep -iE 'pymssql|OpenSSL'
#   YES  pymssql
#   YES  mssql-python
#   YES  no library with its own OpenSSL 1.1 (FIPS)
```

Rebuild with a new tag

1. Change the files: a collection added, a version pinned or raised.
2. Build with the **next tag**, for example `-t site-ee-windows:2` (step 2 of the EE's chapter).
3. Check it with `ee-check`, then push or load it (chapter 6, step 1).
4. In AAP, change the EE's **Image** to `:2` - or run **AAP - set execution environment** with `aap_ee_name` and `aap_ee_image` (chapter 6, step 3). If a job breaks, change it back to `:1`.

Rebuild every few months, even without changes. That is how the Python libraries get their fixes.

Why three collections are pinned

`requirements.yml` pins three collections. Their newest versions need a newer ansible-core than 2.16, and `base-ee:1.0` may have 2.16:

YAML

```
  - name: community.general
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
  - name: community.crypto
    version: "<3.0.0"       # 2.x: the newest for ansible-core 2.16 (3 needs 2.17)
  - name: lowlydba.sqlserver
    version: "<3.0.0"       # 2.x: the newest for ansible-core 2.16 (3 needs 2.19)
```

If `ansible --version` (chapter 2, step 3) shows 2.20, you can remove those `version:` lines. Then build the next tag to get the newest versions.

base-ee:1.0 already has some of these collections

If `base-ee:1.0` already has a collection from `requirements.yml`, the build keeps the version it has. The build upgrades it only when another collection needs a newer one: `community.windows` needs `ansible.windows` 3 or newer, for example. `ee-check` shows the versions you end up with.

A pin can also take a collection back: `base-ee:1.0` may have `community.general` 12, which needs ansible-core 2.17 (the build says `[WARNING]: Collection community.general does not support Ansible version 2.16.19` - about the base's copy), and the build installs 11.x over it. Since 0.33.1 the finished EE has only the installed version's files: before, the base's 12.x files that 11.x does not have stayed next to them (unused, unless a playbook called a module new in 12).

A new base EE

If you build a new base EE, for example `base-ee:1.1`, change the `name:` line in the three `execution-environment.yml` files. Then build the next tag of each EE.

8. When something goes wrong

You see	It means	Do
`localhost/base-ee:1.0: image not known`, or `pull access denied` for `base-ee`	the build machine does not have your base EE	chapter 2, step 3: pull or load it, and check the `name:` line
`Missing Ansible or Ansible Runner for selected Python`	the build looks for Ansible on another Python than the base EE's	chapter 2, step 3: *The Python must match*
`unauthorized` pulling `aap.example.mil/base-ee:1.0` or `registry.redhat.io/...`	podman is not logged in to that registry	`podman login aap.example.mil` (or `registry.redhat.io`)
`bash: /opt/builder/bin/entrypoint: Permission denied`	the image's start script cannot be read by the user it runs as (1000)	the base EE: run its commands with `--entrypoint` (chapter 2, step 3); an EE you built: build the next tag from 0.33.1's `execution-environment.yml`, which makes it readable
`WARN[0000] Failed to mount subscriptions ... permission denied`	podman, run as you, cannot hand this machine's RHEL subscription to the container	nothing, unless the build stops at `No match for argument` or a repository error: then build as root (chapter 2, step 3)
`ee-check`: `no pymssql - its own OpenSSL stops in FIPS mode`, or `pymssql brings its own OpenSSL 1.1 (pymssql.libs)` (before 0.33.1: `Aborted (core dumped)` at `import pymssql`)	the EE has PyPI's ready-made pymssql, which brings its own OpenSSL 1.1.1k: on a FIPS host its self-test fails (`FATAL FIPS SELFTEST FAILURE`) and stops Python	build it again from 0.60.0's files or later (chapter 7, *0.60.0*): they build pymssql from its source, with the EE's own OpenSSL
`additional_build_files ... freetds.tar.gz ... does not exist`	FreeTDS's source is not in the folder	chapter 2, step 7, *And FreeTDS* (or build with `build-ee.sh`, which downloads it)
`freetds.tar.gz is not FreeTDS 1.5.20`	another file, or a download cut short	download it again: chapter 2, step 7
SQL Server with Kerberos: `GSSAPI ... Server not found in Kerberos database`, or `Login failed` / `TDS server connection failed`	the ticket asked for is `MSSQLSvc/<login_host>:<port>`: no such SPN (an IP address or short name as `login_host`, or the SPN not in AD), or no ticket (`kinit` not run in that job)	the server's full DNS name as `login_host`; `setspn -L` on the SQL Server service account shows its SPNs; `klist` in the job after `kinit`
pip: `msal ... requires cryptography<47, but you have cryptography 50...`	Palo Alto's `panos-upgrade-assurance` needs a newer `cryptography` than the base EE's `msal` takes	0.33.1's `requirements.txt` asks for `msal>=1.37.0`, which takes it; or, without Palo Alto in the EE, take `paloaltonetworks.panos` out of `requirements.yml` (and its lines out of `check-list.txt`)
`pasta ... permission denied` or `netavark` errors	rootless podman's network helper is blocked	keep `--network=host` on `podman build`

You see	It means	Do
`additional_build_files ... ansible.cfg ... does not exist`	`ansible.cfg` is not in this folder	chapter 2, step 4
`HTTP Code: 401` or `403` installing collections	no token, or the wrong one	`read -rs TOKEN` again (chapter 2, step 5); a new token: *Automation Content* > *API token*
`HTTP Code: 401` from `galaxy.ansible.com`	a token was sent to Galaxy	in `ansible.cfg`, name the Galaxy server `galaxy` (chapter 2, step 4)
`Could not satisfy the following requirements: * community.general:<12.0.0` (or `community.crypto:<3.0.0`, `lowlydba.sqlserver:<3.0.0`)	your hub has only the newest version, which needs a newer ansible-core than your base EE's	the hub's community repository syncs the older versions too, or `source: https://galaxy.ansible.com` under those three in `requirements.yml` (chapter 2, step 4, *And the versions for ansible-core 2.16*)
`Could not satisfy the following requirements: * containers.podman:*` (any collection, no version)	your hub does not have that collection at all	`FETCH=1 INSECURE=1 bash check-collections.sh all-in-one` lists every one it lacks and downloads them from Galaxy; upload them (chapter 2, step 4)
`Failed to find collection ...`	your hub does not have that collection	ask the hub admin to sync it (chapter 2, step 4), or remove its line from `requirements.yml` and `check-list.txt`
`No match for argument: <package>`	that RHEL package is not in the build machine's repositories	check the subscription (`sudo subscription-manager status`); if a collection wants it only to compile, add it under `exclude: system:` in `execution-environment.yml`
`Could not find a version that satisfies the requirement ...`	the build cannot reach PyPI	add `--build-arg PIP_INDEX_URL=...` (chapter 2, step 7)
pip: `CERTIFICATE_VERIFY_FAILED` or `SSLError`	TLS inspection: pip does not trust the certificate it gets	`--build-arg PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org"` (or your mirror's host), or `INSECURE=1 bash build-ee.sh ...` (chapter 2, step 7)
ansible-galaxy: `SSL: CERTIFICATE_VERIFY_FAILED` installing collections	TLS inspection, or the hub's certificate is not trusted	`--build-arg ANSIBLE_GALAXY_CLI_COLLECTION_OPTS=--ignore-certs`, or `INSECURE=1` (chapter 2, step 7)
podman: `x509: certificate signed by unknown authority`	the registry's certificate is not trusted	`--tls-verify=false` on `podman pull`, `build`, `login`, `push`, or `INSECURE=1`
`Python.h: No such file or directory`	the C headers of the EE's Python are missing	the build installs them from your RHEL repositories: check the subscription
`ee-check` ends with `MISSING`	a library, collection or tool is not in the image	read the build's output for that name; rebuild

You see	It means	Do
a job: `couldn't resolve module/action`	the module's collection is not in the job's EE	the template's **Execution environment**; `ee-check` lists the collections
a job: `Collection ... does not support Ansible version`	the collection needs a newer ansible-core than `base-ee:1.0` has	pin it to an older version: chapter 7, *Why three collections are pinned*
a job: `certificate verify failed`	the EE does not have your CA	put it in `ca/` (chapter 2, step 6), and build the next tag
`kinit: Cannot find KDC for realm "EXAMPLE.MIL"`	the EE cannot find the domain controllers in DNS	use `--network=host` on `podman run`; the domain must be in capitals
a job: `KDC has no support for encryption type`	the FIPS Kerberos setting is not in the image	`ee-check` must say YES to both Kerberos lines: rebuild from these files
a job: `ntlm: ... unsupported`	NTLM on a FIPS host (no MD4)	use Kerberos: the user as `user@EXAMPLE.MIL`
Palo Alto: `Failed connection: URLError` or a timeout	the EE cannot reach the firewall's HTTPS port (443)	firewall rules from the AAP execution nodes to the management interface
Palo Alto: `Invalid credentials`	the user or password is wrong, or the account has no XML API rights	the admin role must allow the XML API
Cisco: `unable to open shell` or `Authentication failed`	SSH or the login failed	test from the EE: `podman run --rm -it --network=host site-ee-network:1 ssh user@switch01`
`lowlydba.sqlserver`: `dbatools` not found	dbatools is not on that SQL Server	chapter 3, step 6: install dbatools

9. Appendix: the build files

The files in the repository's `ee/` folder (and in every release zip), one folder per EE. **Copy them from there**, not from this PDF: a PDF copy can change spaces, and YAML depends on them.

EE 1: Windows, AD and SQL - folder windows/

windows/execution-environment.yml

```
---
# The Windows, Active Directory and SQL execution environment: Windows over WinRM (Kerberos),
  Active
# Directory (and AD as an AAP inventory), SQL Server, MySQL and MariaDB.
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   curl -fo freetds.tar.gz https://www.freetds.org/files/stable/freetds-1.5.20.tar.gz   # for
  pymssql (build-ee.sh does it)
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-windows:1 context
version: 3

images:
  base_image:
    # Your base EE - the one built with the earlier EE guide. This EE adds to everything it has.
    # Pulled from your hub: podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0. None?
      Use AAP's
    # Default EE: registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
    name: localhost/base-ee:1.0

dependencies:
  # The Python that Ansible runs on in your base EE (ansible-builder would take /usr/bin/python3,
    which
  # is RHEL 9's Python 3.9). Check it: podman run --rm localhost/base-ee:1.0 bash -c 'head -1
    $(command -v ansible)'
  python_interpreter:
    python_path: /usr/bin/python3.12
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon (your base EE may have it) asks for these to compile ansible-pylibssh,
        which
      # comes ready-built (a wheel); both are in CodeReady Builder, which a RHEL build does not
        have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: krb5-ad-enctypes.conf
    dest: configs
  - src: check-ee.sh
```

windows/execution-environment.yml (continued)

```
    dest: configs
  - src: check-list.txt
    dest: configs
  - src: ca
    dest: ca
  - src: freetds.tar.gz
    dest: freetds

additional_build_steps:
  prepend_base:
    # pip in every stage of the build reads this build-only file: PyPI or your mirror (--build-arg
    # PIP_INDEX_URL=https://...), and the hosts pip may reach without checking their certificate
      (--build-arg
    # PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org" - behind TLS inspection; build-ee.sh
      INSECURE=1 sets it).
    # The file is removed at the end: the finished EE's pip is as before.
    - ARG PIP_INDEX_URL
    - ARG PIP_TRUSTED_HOST
    - ENV PIP_CONFIG_FILE=/etc/pip-ee-build.conf
    # no-binary: pymssql from its source, never PyPI's ready-made one (prepend_builder says why)
    - RUN { echo '[global]'; echo 'no-binary = pymssql'; [ -z "$PIP_INDEX_URL" ] || echo
      "index-url = $PIP_INDEX_URL"; [ -z "$PIP_TRUSTED_HOST" ] || echo "trusted-host =
      $PIP_TRUSTED_HOST"; } > /etc/pip-ee-build.conf
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    # the C headers of the EE's own Python (gssapi, krb5 and pykerberos compile against them), and
      pip's
    # wheel package: pykerberos (WinRM's Kerberos) is then built here, not in the final image
    - RUN $PKGMGR install -y python$($PYCMD -c 'import sys; print("%d.%d" %
      sys.version_info[:2])')-devel && $PKGMGR clean all
    - RUN $PYCMD -m pip install --no-cache-dir wheel
    # pymssql (SQL Server) built here, against the image's own OpenSSL and Kerberos: PyPI's
      ready-made pymssql brings
    # its own OpenSSL 1.1.1k, which stops Python on a FIPS host (FATAL FIPS SELFTEST FAILURE). Its
      SQL Server client,
    # FreeTDS 1.5.20 (freetds.tar.gz, from www.freetds.org - build-ee.sh fetches it), with
      Kerberos, built into
    # pymssql itself: nothing of FreeTDS stays in the image
    - COPY _build/freetds/freetds.tar.gz /tmp/freetds.tar.gz
    - RUN echo "2e4d2d50840c8d4423aa9df7ef5de8b5f1f1881f72d402ed70c4ea55a464e31e
      /tmp/freetds.tar.gz" | sha256sum -c - || { echo "freetds.tar.gz is not FreeTDS 1.5.20 - get
      it with curl -fo freetds.tar.gz https://www.freetds.org/files/stable/freetds-1.5.20.tar.gz"
      >&2; exit 1; }
```

windows/execution-environment.yml (continued)

```
    - RUN $PKGMGR install -y gcc make tar gzip openssl-devel krb5-devel && $PKGMGR clean all
    - RUN mkdir /tmp/freetds && tar -xzf /tmp/freetds.tar.gz -C /tmp/freetds --strip-components 1
      && cd /tmp/freetds && CFLAGS="-fPIC -O2" ./configure --prefix=/opt/freetds --with-tdsver=7.4
      --enable-msdblib --enable-krb5 --with-openssl --with-gnutls=no --disable-odbc
      --disable-server --disable-pool --disable-apps --enable-static --disable-shared > /dev/null
      && make -j"$(nproc)" > /dev/null && make install > /dev/null && cd / && rm -rf /tmp/freetds
      /tmp/freetds.tar.gz
    - ENV PYMSSQL_FREETDS=/opt/freetds
  prepend_final:
    # the collections exactly as the galaxy stage left them: copied on top of the base EE's own,
      the files of a
    # newer version a pin replaces would stay (community.general 12's next to 11.4.9's)
    - RUN rm -rf /usr/share/ansible/collections/ansible_collections
  append_final:
    # Kerberos to Active Directory from a FIPS host: AD's AES-SHA1 keys allowed
      (krb5-ad-enctypes.conf)
    - COPY _build/configs/krb5-ad-enctypes.conf /etc/krb5.conf.d/ad-enctypes.conf
    - RUN rm -f /etc/krb5.conf.d/crypto-policies
    # Kerberos tickets in a file: the kernel keyring RHEL uses by default does not work in a
      container
    - ENV KRB5CCNAME=FILE:/tmp/krb5cc_ee
    # your CA certificates (ca/*.pem) in the trust store - LDAPS, SQL Server TLS, AAP
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: does the image have what it should (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - COPY _build/configs/check-list.txt /usr/local/share/ee-check.list
    # every file copied in readable by the user jobs run as (1000), the entrypoint every job
      starts through too,
    # whatever modes the build machine gave them - a start script user 1000 cannot read: "bash:
      /opt/builder/bin/
    # entrypoint: Permission denied"
    - RUN chmod 755 /opt/builder/bin/entrypoint /usr/local/bin/ee-check && chmod 644
      /usr/local/share/ee-check.list /etc/krb5.conf.d/ad-enctypes.conf
    # the build-only pip configuration (prepend_base) out of the finished image
    - RUN rm -f /etc/pip-ee-build.conf
```

windows/requirements.yml

```
---
# The collections of the Windows, Active Directory and SQL EE. "certified" = Red Hat Automation
  Hub (also in a
# Private Automation Hub that syncs it); "community" = Galaxy (a Private Automation Hub needs its
  community
# repository to have it). ansible-builder reads each collection's own Python and system needs.
# After the first good build, pin each one to the version it got: podman run --rm <image> ee-check
collections:
  # ---- Windows ----
  - name: ansible.windows               # certified: the core Windows modules (win_service,
    win_updates ...)
  - name: community.windows             # community: more Windows modules
  # ---- Active Directory ----
  - name: microsoft.ad                  # certified: users, groups, computers, OUs - and AD as an
    AAP inventory
  # ---- SQL Server ----
  - name: community.general             # community: mssql_db, mssql_script (SQL Server over port
    1433)
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
  - name: lowlydba.sqlserver            # community: SQL Server through dbatools on a Windows host
    (logins, jobs, backups)
    version: "<3.0.0"      # 2.x: the newest for ansible-core 2.16 (3 needs 2.19)
  # ---- MySQL and MariaDB ----
  - name: ansible.mysql                 # MySQL and MariaDB (was community.mysql)
```

windows/requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs). They
  come from PyPI.
# ---- Windows over WinRM ----
pywinrm[kerberos]
requests-kerberos
requests-ntlm          # NTLM: not on a FIPS host (no MD4); Kerberos is the way there
pypsrp[kerberos]       # the psrp connection, an alternative to winrm
# ---- Active Directory over LDAP, from the EE (microsoft.ad brings sansldap, pyspnego, dnspython)
  ----
ldap3                  # pure Python: your own LDAP scripts
# ---- SQL Server (community.general.mssql_db, mssql_script) - ansible.mysql brings PyMySQL itself
  ----
pymssql                # built from its source, with the EE's own OpenSSL: PyPI's ready-made one
  stops Python on a
                       # FIPS host (execution-environment.yml, prepend_builder; freetds.tar.gz)
mssql-python           # Microsoft's SQL Server driver, for your own Python (import mssql_python;
  the mssql_* modules
                       # use pymssql): its ODBC Driver 18 is in the wheel, Kerberos through the
                         system's krb5
# ---- general ----
jmespath               # the json_query filter
dnspython              # the community.general.dig lookup
smbprotocol            # the report website on an SMB share (report_web_dest \\server\share):
  Kerberos through pyspnego
```

windows/bindep.txt

```
# System packages (RPMs, from Red Hat) the collections do not bring themselves (ansible-builder
  adds theirs).
krb5-workstation [platform:rpm]       # kinit / klist: check Kerberos inside the EE
bind-utils [platform:rpm]             # dig / nslookup: check DNS (the domain controllers' SRV
  records)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
```

windows/check-list.txt

```
# What ee-check looks for in this EE: lib <python import> <package>, collection <name>, tool
  <command>
name Windows, Active Directory and SQL EE
# ---- Windows over WinRM ----
lib winrm pywinrm
lib kerberos pykerberos
lib requests_kerberos requests-kerberos
lib requests_ntlm requests-ntlm
lib pypsrp pypsrp
# ---- Active Directory ----
lib spnego pyspnego
lib gssapi gssapi
lib krb5 krb5
lib sansldap sansldap
lib ldap3 ldap3
lib dns dnspython
# ---- SQL ----
lib pymssql pymssql
lib mssql_python mssql-python
lib pymysql PyMySQL
# ---- general ----
lib jmespath jmespath
lib smbclient smbprotocol
lib cryptography cryptography
collection ansible.windows
collection community.windows
collection microsoft.ad
collection community.general
collection lowlydba.sqlserver
collection ansible.mysql
tool kinit
tool klist
tool dig
tool ping
kerberos
```

EE 2: Network (Cisco and Palo Alto) - folder network/

network/execution-environment.yml

```
---
# The network execution environment: Cisco (IOS / IOS-XE, Nexus, ASA) and Palo Alto (PAN-OS
  firewalls and
# Panorama).
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-network:1 context
version: 3

images:
  base_image:
    # Your base EE - the one built with the earlier EE guide. This EE adds to everything it has.
    # Pulled from your hub: podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0. None?
      Use AAP's
    # Default EE: registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
    name: localhost/base-ee:1.0

dependencies:
  # The Python that Ansible runs on in your base EE (ansible-builder would take /usr/bin/python3,
    which
  # is RHEL 9's Python 3.9). Check it: podman run --rm localhost/base-ee:1.0 bash -c 'head -1
    $(command -v ansible)'
  python_interpreter:
    python_path: /usr/bin/python3.12
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon asks for these to compile ansible-pylibssh, which comes ready-built (a
        wheel);
      # both are in the CodeReady Builder repository, which a RHEL build does not have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: check-ee.sh
    dest: configs
  - src: check-list.txt
    dest: configs
  - src: ca
    dest: ca
```

network/execution-environment.yml (continued)

```
additional_build_steps:
  prepend_base:
    # pip in every stage of the build reads this build-only file: PyPI or your mirror (--build-arg
    # PIP_INDEX_URL=https://...), and the hosts pip may reach without checking their certificate
      (--build-arg
    # PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org" - behind TLS inspection; build-ee.sh
      INSECURE=1 sets it).
    # The file is removed at the end: the finished EE's pip is as before.
    - ARG PIP_INDEX_URL
    - ARG PIP_TRUSTED_HOST
    - ENV PIP_CONFIG_FILE=/etc/pip-ee-build.conf
    - RUN { echo '[global]'; [ -z "$PIP_INDEX_URL" ] || echo "index-url = $PIP_INDEX_URL"; [ -z
      "$PIP_TRUSTED_HOST" ] || echo "trusted-host = $PIP_TRUSTED_HOST"; } > /etc/pip-ee-build.conf
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    - RUN $PYCMD -m pip install --no-cache-dir wheel
  prepend_final:
    # the collections exactly as the galaxy stage left them: copied on top of the base EE's own,
      the files of a
    # newer version a pin replaces would stay (community.general 12's next to 11.4.9's)
    - RUN rm -rf /usr/share/ansible/collections/ansible_collections
  append_final:
    # your CA certificates (ca/*.pem) in the trust store - Panorama, the firewalls' HTTPS, AAP
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: does the image have what it should (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - COPY _build/configs/check-list.txt /usr/local/share/ee-check.list
    # every file copied in readable by the user jobs run as (1000), the entrypoint every job
      starts through too,
    # whatever modes the build machine gave them - a start script user 1000 cannot read: "bash:
      /opt/builder/bin/
    # entrypoint: Permission denied"
    - RUN chmod 755 /opt/builder/bin/entrypoint /usr/local/bin/ee-check && chmod 644
      /usr/local/share/ee-check.list
    # the build-only pip configuration (prepend_base) out of the finished image
    - RUN rm -f /etc/pip-ee-build.conf
```

network/requirements.yml

```
---
# The collections of the network EE. "certified" = Red Hat Automation Hub (also in a Private
  Automation Hub
# that syncs it); "partner" = made by the vendor (in your hub's rh-certified repository when it
  has it -
# otherwise Galaxy). ansible-builder reads each collection's own Python and system needs.
# After the first good build, pin each one to the version it got: podman run --rm <image> ee-check
# Only one vendor? Remove the other's lines here and in check-list.txt.
collections:
  # ---- the network connections, and helpers ----
  - name: ansible.netcommon             # certified: network_cli (SSH), netconf, httpapi
    connections
  - name: ansible.utils                 # certified: ipaddr filters, cli_parse, validate
  # ---- Cisco ----
  - name: cisco.ios                     # certified: IOS / IOS-XE
  - name: cisco.nxos                    # certified: Nexus
  - name: cisco.asa                     # certified: ASA firewalls
  # ---- Palo Alto ----
  - name: paloaltonetworks.panos        # partner: PAN-OS firewalls and Panorama (over their XML
    API, HTTPS)
```

network/requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs). They
  come from PyPI.
# The collections bring the rest: ansible-pylibssh, paramiko, ncclient (Cisco); pan-os-python,
  pan-python,
# xmltodict (Palo Alto); netaddr, textfsm, ttp (ansible.utils).
jmespath               # the json_query filter
msal>=1.37.0           # base-ee:1.0 has msal (Azure logins) asking for cryptography<47; Palo
  Alto's panos-upgrade-assurance
                       # 2.1.1 needs pyOpenSSL 26, so cryptography 49+: msal 1.37 and later take
                         it (no pip conflict)
smbprotocol            # the report website on an SMB share (report_web_dest \\server\share):
  Kerberos through pyspnego
```

network/bindep.txt

```
# System packages (RPMs, from Red Hat) the collections do not bring themselves (ansible-builder
  adds theirs).
openssh-clients [platform:rpm]        # ssh / scp: test a device's SSH from inside the EE
sshpass [platform:rpm]                # password SSH (network devices that need it)
git-core [platform:rpm]               # git: the network config backup's repository
  (net_config_store: git)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
bind-utils [platform:rpm]             # dig / nslookup
```

network/check-list.txt

```
# What ee-check looks for in this EE: lib <python import> <package>, collection <name>, tool
  <command>
name Network EE (Cisco and Palo Alto)
# ---- Cisco and the network connections ----
lib pylibsshext ansible-pylibssh
lib paramiko paramiko
lib ncclient ncclient
lib netaddr netaddr
lib textfsm textfsm
lib ttp ttp
# ---- Palo Alto ----
lib panos pan-os-python
lib pan.xapi pan-python
lib panos_upgrade_assurance panos-upgrade-assurance
lib xmltodict xmltodict
# ---- general ----
lib jmespath jmespath
lib smbclient smbprotocol
collection ansible.netcommon
collection ansible.utils
collection cisco.ios
collection cisco.nxos
collection cisco.asa
collection paloaltonetworks.panos
tool ssh
tool sshpass
tool ping
tool git
tool dig
```

EE 3: All-in-one - folder all-in-one/

all-in-one/execution-environment.yml

```
---
# The all-in-one execution environment: Linux, Windows and Active Directory, VMware, NetApp,
  databases
# (MySQL, MariaDB, SQL Server), Cisco, Palo Alto, ServiceNow - and every site-automation job.
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   curl -fo freetds.tar.gz https://www.freetds.org/files/stable/freetds-1.5.20.tar.gz   # for
  pymssql (build-ee.sh does it)
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-all:1 context
version: 3

images:
  base_image:
    # Your base EE - the one built with the earlier EE guide. This EE adds to everything it has.
    # Pulled from your hub: podman tag aap.example.mil/base-ee:1.0 localhost/base-ee:1.0. None?
      Use AAP's
    # Default EE: registry.redhat.io/ansible-automation-platform-27/ee-supported-rhel9:latest
    name: localhost/base-ee:1.0

dependencies:
  # The Python that Ansible runs on in your base EE (ansible-builder would take /usr/bin/python3,
    which
  # is RHEL 9's Python 3.9). Check it: podman run --rm localhost/base-ee:1.0 bash -c 'head -1
    $(command -v ansible)'
  python_interpreter:
    python_path: /usr/bin/python3.12
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon asks for these to compile ansible-pylibssh, which comes ready-built (a
        wheel);
      # both are in the CodeReady Builder repository, which a RHEL build does not have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: krb5-ad-enctypes.conf
    dest: configs
  - src: check-ee.sh
    dest: configs
```

all-in-one/execution-environment.yml (continued)

```
  - src: check-list.txt
    dest: configs
  - src: ca
    dest: ca
  - src: freetds.tar.gz
    dest: freetds

additional_build_steps:
  prepend_base:
    # pip in every stage of the build reads this build-only file: PyPI or your mirror (--build-arg
    # PIP_INDEX_URL=https://...), and the hosts pip may reach without checking their certificate
      (--build-arg
    # PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org" - behind TLS inspection; build-ee.sh
      INSECURE=1 sets it).
    # The file is removed at the end: the finished EE's pip is as before.
    - ARG PIP_INDEX_URL
    - ARG PIP_TRUSTED_HOST
    - ENV PIP_CONFIG_FILE=/etc/pip-ee-build.conf
    # no-binary: pymssql from its source, never PyPI's ready-made one (prepend_builder says why)
    - RUN { echo '[global]'; echo 'no-binary = pymssql'; [ -z "$PIP_INDEX_URL" ] || echo
      "index-url = $PIP_INDEX_URL"; [ -z "$PIP_TRUSTED_HOST" ] || echo "trusted-host =
      $PIP_TRUSTED_HOST"; } > /etc/pip-ee-build.conf
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    # the C headers of the EE's own Python (gssapi, krb5 and pykerberos compile against them), and
      pip's
    # wheel package: pykerberos (WinRM's Kerberos) is then built here, not in the image, which has
      no compiler
    - RUN $PKGMGR install -y python$($PYCMD -c 'import sys; print("%d.%d" %
      sys.version_info[:2])')-devel && $PKGMGR clean all
    - RUN $PYCMD -m pip install --no-cache-dir wheel
    # pymssql (SQL Server) built here, against the image's own OpenSSL and Kerberos: PyPI's
      ready-made pymssql brings
    # its own OpenSSL 1.1.1k, which stops Python on a FIPS host (FATAL FIPS SELFTEST FAILURE). Its
      SQL Server client,
    # FreeTDS 1.5.20 (freetds.tar.gz, from www.freetds.org - build-ee.sh fetches it), with
      Kerberos, built into
    # pymssql itself: nothing of FreeTDS stays in the image
    - COPY _build/freetds/freetds.tar.gz /tmp/freetds.tar.gz
    - RUN echo "2e4d2d50840c8d4423aa9df7ef5de8b5f1f1881f72d402ed70c4ea55a464e31e
      /tmp/freetds.tar.gz" | sha256sum -c - || { echo "freetds.tar.gz is not FreeTDS 1.5.20 - get
      it with curl -fo freetds.tar.gz https://www.freetds.org/files/stable/freetds-1.5.20.tar.gz"
      >&2; exit 1; }
```

all-in-one/execution-environment.yml (continued)

```
    - RUN $PKGMGR install -y gcc make tar gzip openssl-devel krb5-devel && $PKGMGR clean all
    - RUN mkdir /tmp/freetds && tar -xzf /tmp/freetds.tar.gz -C /tmp/freetds --strip-components 1
      && cd /tmp/freetds && CFLAGS="-fPIC -O2" ./configure --prefix=/opt/freetds --with-tdsver=7.4
      --enable-msdblib --enable-krb5 --with-openssl --with-gnutls=no --disable-odbc
      --disable-server --disable-pool --disable-apps --enable-static --disable-shared > /dev/null
      && make -j"$(nproc)" > /dev/null && make install > /dev/null && cd / && rm -rf /tmp/freetds
      /tmp/freetds.tar.gz
    - ENV PYMSSQL_FREETDS=/opt/freetds
  prepend_final:
    # the collections exactly as the galaxy stage left them: copied on top of the base EE's own,
      the files of a
    # newer version a pin replaces would stay (community.general 12's next to 11.4.9's)
    - RUN rm -rf /usr/share/ansible/collections/ansible_collections
  append_final:
    # Kerberos to Active Directory from a FIPS host: AD's AES-SHA1 keys allowed
      (krb5-ad-enctypes.conf)
    - COPY _build/configs/krb5-ad-enctypes.conf /etc/krb5.conf.d/ad-enctypes.conf
    - RUN rm -f /etc/krb5.conf.d/crypto-policies
    # Kerberos tickets in a file: the kernel keyring RHEL uses by default does not work in a
      container
    - ENV KRB5CCNAME=FILE:/tmp/krb5cc_ee
    # your CA certificates (ca/*.pem) in the trust store - vCenter, ServiceNow, AAP, LDAPS, ONTAP,
      Panorama
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: does the image have what it should (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - COPY _build/configs/check-list.txt /usr/local/share/ee-check.list
    # every file copied in readable by the user jobs run as (1000), the entrypoint every job
      starts through too,
    # whatever modes the build machine gave them - a start script user 1000 cannot read: "bash:
      /opt/builder/bin/
    # entrypoint: Permission denied"
    - RUN chmod 755 /opt/builder/bin/entrypoint /usr/local/bin/ee-check && chmod 644
      /usr/local/share/ee-check.list /etc/krb5.conf.d/ad-enctypes.conf
    # the build-only pip configuration (prepend_base) out of the finished image
    - RUN rm -f /etc/pip-ee-build.conf
```

all-in-one/requirements.yml

```
---
# The Ansible collections of the all-in-one execution environment (ee/all-in-one,
  docs/EXECUTION_ENVIRONMENT.md).
# "certified" = Red Hat Automation Hub (also in a Private Automation Hub that syncs it);
  "community" = Galaxy;
# "partner" = made by the vendor (in your hub's rh-certified repository when it has it - otherwise
  Galaxy)
# (a Private Automation Hub needs its community repository synced for these). ansible-builder
  reads each
# collection's own Python and system needs: they are not repeated in requirements.txt /
  bindep.txt.
# After the first good build, pin each one to the version it got: podman run --rm <image> ee-check
collections:
  # ---- Linux ----
  - name: ansible.posix                # certified: acl, firewalld, mount, sysctl, selinux ...
  - name: ansible.utils                # certified: ipaddr filters, validate, cli_parse
  - name: community.general            # community: hundreds of modules (ldap_*, mssql_*, nmcli,
    ...)
    version: "<12.0.0"      # 11.x: the newest for ansible-core 2.16 (12 needs 2.17)
  - name: community.crypto             # community: certificates, CSRs, keys
    version: "<3.0.0"      # 2.x: the newest for ansible-core 2.16 (3 needs 2.17)
  - name: containers.podman            # community: podman containers, pods, Quadlet
  - name: redhat.rhel_system_roles     # certified (Automation Hub only): RHEL System Roles
  # ---- Windows and Active Directory ----
  - name: ansible.windows              # certified: the core Windows modules
  - name: microsoft.ad                 # certified: AD objects, users, groups - and the LDAP
    inventory plugin
  - name: community.windows            # community: more Windows modules
  # ---- VMware ----
  - name: vmware.vmware                # certified: vCenter (site-automation's VMware jobs need
    >= 2.11)
    version: ">=2.11.0"
  # ---- NetApp ----
  - name: netapp.ontap                 # certified: ONTAP REST modules
  # ---- Databases ----
  - name: ansible.mysql                # certified: MySQL and MariaDB (was community.mysql)
  - name: lowlydba.sqlserver           # community: SQL Server through dbatools on a Windows host
    (logins, jobs, backups)
    version: "<3.0.0"      # 2.x: the newest for ansible-core 2.16 (3 needs 2.19)
  # ---- Cisco and network ----
  - name: ansible.netcommon            # certified: network_cli, netconf, httpapi connections
  - name: cisco.ios                    # certified: IOS / IOS-XE
  - name: cisco.nxos                   # certified: Nexus
  - name: cisco.asa                    # certified: ASA firewalls
  # ---- Palo Alto ----
  - name: paloaltonetworks.panos       # partner: PAN-OS firewalls and Panorama (over their XML
    API, HTTPS)
  # ---- ServiceNow ----
```

all-in-one/requirements.yml (continued)

```
  - name: servicenow.itsm              # certified: tickets, and the CMDB inventory plugin
    (servicenow.itsm.now)
```

all-in-one/requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs).
# Windows over WinRM (ansible.windows, community.windows, microsoft.ad modules, site-automation's
  Windows jobs)
pywinrm[kerberos]
requests-kerberos
requests-ntlm          # NTLM: not on a FIPS host (no MD4); Kerberos is the way there
pypsrp[kerberos]       # the psrp connection, an alternative to winrm
# Active Directory from the EE itself, over LDAP (microsoft.ad brings sansldap, pyspnego,
  dnspython)
ldap3                  # pure Python: your own scripts, no compiler
# SQL Server (community.general.mssql_db, mssql_script): built from its source, against the EE's
  own OpenSSL - PyPI's
# ready-made one stops Python on a FIPS host (execution-environment.yml, prepend_builder;
  freetds.tar.gz)
pymssql
mssql-python           # Microsoft's SQL Server driver, for your own Python (import mssql_python;
  the mssql_* modules
                       # use pymssql): its ODBC Driver 18 is in the wheel, Kerberos through the
                         system's krb5
# Linux and general
jmespath               # the json_query filter
dnspython              # the community.general.dig lookup
msal>=1.37.0           # base-ee:1.0 has msal (Azure logins) asking for cryptography<47; Palo
  Alto's panos-upgrade-assurance
                       # 2.1.1 needs pyOpenSSL 26, so cryptography 49+: msal 1.37 and later take
                         it (no pip conflict)
smbprotocol            # the report website on an SMB share (report_web_dest \\server\share):
  Kerberos through pyspnego
```

all-in-one/bindep.txt

```
# System packages (RPMs) the collections do not bring themselves (ansible-builder adds theirs).
# [compile] = only while building; the rest stays in the image.
krb5-workstation [platform:rpm]       # kinit / klist: check Kerberos inside the EE
openssh-clients [platform:rpm]
sshpass [platform:rpm]                # password SSH (network devices that need it)
git-core [platform:rpm]               # git: the network config backup's repository
  (net_config_store: git)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
bind-utils [platform:rpm]             # dig / nslookup: check DNS (the domain controllers' SRV
  records)
tzdata [platform:rpm]                 # the time zones: report_time_zone (0.32.0), the VMware
  reports in your time zone
```

all-in-one/check-list.txt

```
# What ee-check looks for in this EE: lib <python import> <package>, collection <name>, tool
  <command>
name All-in-one EE
# ---- Windows over WinRM ----
lib winrm pywinrm
lib kerberos pykerberos
lib requests_kerberos requests-kerberos
lib requests_ntlm requests-ntlm
lib pypsrp pypsrp
# ---- Active Directory ----
lib spnego pyspnego
lib gssapi gssapi
lib krb5 krb5
lib sansldap sansldap
lib ldap3 ldap3
lib dns dnspython
# ---- VMware ----
lib pyVmomi pyVmomi
lib com.vmware.vcenter_client vmware-vcenter
# ---- NetApp, ServiceNow ----
lib requests requests
lib solidfire solidfire-sdk-python
# ---- SQL ----
lib pymssql pymssql
lib mssql_python mssql-python
lib pymysql PyMySQL
# ---- Cisco and the network connections ----
lib pylibsshext ansible-pylibssh
lib paramiko paramiko
lib ncclient ncclient
lib netaddr netaddr
lib textfsm textfsm
# ---- Palo Alto ----
lib panos pan-os-python
lib pan.xapi pan-python
lib xmltodict xmltodict
# ---- general ----
lib jmespath jmespath
lib smbclient smbprotocol
lib cryptography cryptography
collection ansible.posix
collection ansible.utils
collection community.general
collection community.crypto
collection containers.podman
collection redhat.rhel_system_roles
collection ansible.windows
collection microsoft.ad
```

all-in-one/check-list.txt (continued)

```
collection community.windows
collection vmware.vmware
collection netapp.ontap
collection ansible.mysql
collection lowlydba.sqlserver
collection ansible.netcommon
collection cisco.ios
collection cisco.nxos
collection cisco.asa
collection paloaltonetworks.panos
collection servicenow.itsm
tool kinit
tool klist
tool ssh
tool sshpass
tool dig
tool ping
tool git
kerberos
```

The same in every folder

ansible.cfg.example (in all three folders)

```
# Copy to ansible.cfg (this folder) when the collections come from your automation hub. The URLs:
  AAP >
# Automation Content > Repositories > a repository > "Copy CLI configuration" (AAP 2.5 and later:
  the
# platform's own address). The token is NOT written here: it is given at build time (--build-arg).
[galaxy]
server_list = rh_certified, community, published

[galaxy_server.rh_certified]
url = https://aap.example.mil/api/galaxy/content/rh-certified/

[galaxy_server.community]
url = https://aap.example.mil/api/galaxy/content/community/

[galaxy_server.published]
url = https://aap.example.mil/api/galaxy/content/published/
```

check-ee.sh (in all three folders)

```
#!/bin/bash
# ee-check: does this execution environment have what it should? Run it inside the image:
#   podman run --rm <image> ee-check
# What it looks for is in /usr/local/share/ee-check.list (the check-list.txt of the EE's folder).
# It ends with "ALL YES" - or with what is missing (and exit code 1).
list=/usr/local/share/ee-check.list
missing=0
yes() { echo "  YES  $1"; }
no()  { echo "  no   $1"; missing=$((missing + 1)); }
# the Python Ansible runs on (the first line of the ansible command), not the system's python3
py=$(head -1 "$(command -v ansible)" 2>/dev/null | sed -e 's/^#! *//' -e 's/ .*//')
[ -x "$py" ] || py=python3
echo "== $(sed -n 's/^name //p' $list)"
echo "== Python: $($py --version 2>&1)   ansible-core: $(ansible --version 2>/dev/null | head -1)"
echo "== Python libraries"
while read -r kind a b; do
  [ "$kind" = lib ] || continue
  # the error caught, a crash too (no "Aborted (core dumped)" on the screen): said in words when
    it is known
  if err=$( { $py -c "import $a" >/dev/null; } 2>&1 ); then yes "$b"
  elif [[ $err == *"FIPS SELFTEST FAILURE"* ]]; then no "$b - its own OpenSSL stops in FIPS mode
    (EXECUTION_ENVIRONMENT.md, chapter 8)"
  else no "$b"; fi
done < $list
# a library that brings its own OpenSSL 1.1 (PyPI's ready-made pymssql): on a FIPS host it stops
  Python
own=0
for d in $($py -c 'import site; print(" ".join(site.getsitepackages()))' 2>/dev/null); do
  for f in "$d"/*.libs/libcrypto-*.so.1.1*; do
    [ -e "$f" ] || continue
    own=1; f=${f%/*}; f=${f##*/}; no "${f%.libs} brings its own OpenSSL 1.1 ($f), which stops
      Python on a FIPS host: build the EE again from the current ee/ files
      (EXECUTION_ENVIRONMENT.md, chapter 8)"
  done
done
[ "$own" = 1 ] || yes "no library with its own OpenSSL 1.1 (FIPS)"
echo "== Collections"
have=$(ansible-galaxy collection list 2>/dev/null)
while read -r kind a _; do
  [ "$kind" = collection ] || continue
  v=$(echo "$have" | awk -v c="$a" '$1 == c { print $2; exit }')
  if [ -n "$v" ]; then yes "$a $v"; else no "$a"; fi
done < $list
echo "== Tools"
while read -r kind a _; do
  [ "$kind" = tool ] || continue
  if command -v "$a" >/dev/null; then yes "$a"; else no "$a"; fi
done < $list
```

check-ee.sh (in all three folders) (continued)

```
if grep -q '^kerberos' $list; then
  echo "== Kerberos (FIPS, Active Directory)"
  if [ -e /etc/krb5.conf.d/crypto-policies ]; then no "crypto-policies link removed (AD's AES-SHA1
    keys are refused on a FIPS host)"
  else yes "crypto-policies link removed"; fi
  if grep -qs 'aes256-sha1' /etc/krb5.conf.d/ad-enctypes.conf; then yes "AD encryption types
    allowed (ad-enctypes.conf)"
  else no "AD encryption types allowed (ad-enctypes.conf)"; fi
fi
echo "== Your CA certificates added: $(ls /etc/pki/ca-trust/source/anchors/ 2>/dev/null | wc -l)"
if [ "$missing" -eq 0 ]; then echo "== ALL YES"; else echo "== $missing MISSING - see the lines
  with 'no'"; exit 1; fi
```

krb5-ad-enctypes.conf (in windows and all-in-one)

```
# Kerberos to Active Directory from a FIPS host (STIG). The EE's Kerberos would follow the host's
  FIPS
# policy and offer only AES with SHA-2, while Active Directory's keys are AES with SHA-1: "KDC has
  no
# support for encryption type". This allows both AES kinds - never RC4 or DES.
[libdefaults]
  permitted_enctypes = aes256-sha2 aes128-sha2 aes256-sha1 aes128-sha1
```

The scripts - folder ee/

build-ee.sh

```
#!/usr/bin/env bash
# Build one execution environment of this folder in one go - the steps of
  docs/EXECUTION_ENVIRONMENT.md: first
# the checks of chapter 2 (it stops at the first problem and says which step fixes it), then
  ansible-builder and
# podman build with your hub token, ee-check, and - when you ask - the image pushed to your hub,
  or saved as a
# file to load on the execution nodes. Run it on the build machine, in the folder that holds
  windows/, network/
# and all-in-one/:
#   bash build-ee.sh all-in-one 2                         # builds localhost/site-ee-all:2
#   PUSH=aap.example.mil bash build-ee.sh all-in-one 2    # ... and pushes it:
  aap.example.mil/site-ee-all:2
#   SAVE=1 bash build-ee.sh all-in-one 2                  # ... and saves it: site-ee-all-2.tar
  (no registry)
#   PIP_INDEX_URL=https://nexus.example.mil/repository/pypi/simple bash build-ee.sh ...   # your
  PyPI mirror
#   INSECURE=1 bash build-ee.sh all-in-one 2              # behind TLS inspection: no certificate
  checked - pip
#       --trusted-host (PyPI's hosts, or your mirror's), ansible-galaxy --ignore-certs, podman
  --tls-verify=false
#   PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org" bash build-ee.sh ...   # only pip's
  (INSECURE=1 sets it)
#   sudo env INSECURE=1 bash build-ee.sh all-in-one 2     # as root: when podman cannot give the
  build your RHEL
#       subscription as you (it says so) - the base image in root's podman first: podman save ...
  | sudo podman load
# The hub token: it asks for it (nothing shows as you paste), or takes TOKEN from the environment.
  It is never
# written to a file or shown.
set -euo pipefail
cd "$(dirname "$0")"

folder=${1:-}
tag=${2:-1}
case "$folder" in
  windows)    name=site-ee-windows; aap="Windows, AD and SQL EE" ;;
  network)    name=site-ee-network; aap="Network EE (Cisco and Palo Alto)" ;;
  all-in-one) name=site-ee-all;     aap="All-in-one EE" ;;
  *) echo "usage: bash build-ee.sh windows|network|all-in-one [tag]   (e.g. bash build-ee.sh
    all-in-one 2)" >&2; exit 2 ;;
esac
[[ "$tag" =~ ^[A-Za-z0-9._-]+$ ]] || { echo "the tag is a version such as 2 or 2.1, not '$tag'"
  >&2; exit 2; }
image="localhost/$name:$tag"
curl_k=(); tls=()
if [ -n "${INSECURE:-}" ]; then
  curl_k=(-k); tls=(--tls-verify=false)
```

build-ee.sh (continued)

```
  if [ -z "${PIP_TRUSTED_HOST:-}" ]; then
    if [ -n "${PIP_INDEX_URL:-}" ]; then h=${PIP_INDEX_URL#*://}; h=${h%%/*};
      PIP_TRUSTED_HOST=${h##*@}
    else PIP_TRUSTED_HOST="pypi.org files.pythonhosted.org"; fi
  fi
fi
step() { echo; echo "== $1"; }
ok()   { echo "   ok    $1"; }
note() { echo "   note  $1"; }
stop() { echo "   STOP  $1" >&2; exit 1; }
err=$(mktemp); trap 'rm -f "$err"' EXIT

step "Checks (docs/EXECUTION_ENVIRONMENT.md, chapter 2)"
for t in podman ansible-builder curl; do
  command -v "$t" >/dev/null || stop "$t is not installed: sudo dnf install -y podman
    ansible-builder (chapter 2, step 1)"
done
ok "podman, ansible-builder"
yml=$folder/execution-environment.yml
[ -f "$yml" ] || stop "no $yml here: run it in the folder with windows/, network/ and all-in-one/
  (chapter 2, step 2)"
cfg=$folder/ansible.cfg
[ -f "$cfg" ] || stop "no $cfg: cp $folder/ansible.cfg.example $cfg and put your hub's address in
  it (chapter 2, step 4)"
! grep -q 'aap\.example\.mil' "$cfg" || stop "$cfg still says aap.example.mil: put your hub's
  address in it (chapter 2, step 4)"
hub=$(sed -n 's/^url *= *//p' "$cfg" | head -1)
ok "$cfg: the collections from $hub"

base=$(awk '/base_image:/ {b = 1} b && /name:/ {print $2; exit}' "$yml")
want=$(awk '/python_path:/ {print $2; exit}' "$yml")
if ! podman image exists "$base"; then
  case "$base" in
    localhost/*) stop "the base image $base is not on this machine: podman pull or podman load it,
      then podman tag it $base (chapter 2, step 3)" ;;
    *) podman pull -q "${tls[@]}" "$base" >/dev/null || stop "could not pull the base image $base:
      podman login ${base%%/*} first (chapter 2, step 3)" ;;
  esac
fi
# past the base's entrypoint: in some base images the user they run as cannot read it - the EE
  built here gets a
# readable one (execution-environment.yml)
got=$(podman run --rm --entrypoint bash "$base" -c 'head -1 "$(command -v ansible)"' 2>"$err" |
  sed -e 's/^#! *//' -e 's/ .*//') || true
[ -n "$got" ] || { sed 's/^/   /' "$err" >&2; stop "could not see which Python Ansible runs on in
  $base (chapter 2, step 3)"; }
if [ -n "$want" ] && [ "$got" != "$want" ]; then
  stop "Ansible in $base runs on $got, $yml says $want: sed -i 's|$want|$got|'
```

build-ee.sh (continued)

```
    */execution-environment.yml (chapter 2, step 3)"
fi
ok "the base image $base, its Ansible on $got"
case "$(podman run --rm "$base" true 2>&1 || true)" in
  *"entrypoint: Permission denied"*)
    note "$base's entrypoint cannot be read by the user EEs run as: the EE built here gets a
      readable one" ;;
esac
if grep -q 'Failed to mount subscriptions' "$err" && [ "$(id -u)" != 0 ]; then
  note "podman cannot give the build this machine's RHEL subscription as $(id -un) (WARN ...
    Failed to mount subscriptions):"
  note "the system packages can come only from $base's own repositories. If the build stops at 'No
    match for argument'"
  note "or a repository error, build as root: podman save $base | sudo podman load, then sudo env
    ${INSECURE:+INSECURE=1 }bash build-ee.sh $folder $tag (chapter 8)"
fi

n=$(find "$folder/ca" -maxdepth 1 -name '*.pem' 2>/dev/null | wc -l)
if [ "$n" -gt 0 ]; then ok "$n CA certificate(s) in $folder/ca"
else note "no CA certificate in $folder/ca: the EE will not trust your CA (chapter 2, step 6)"; fi

# curl's own words in the STOP: a certificate (TLS inspection), a name that does not resolve, no
  way out
tls_hint=$([ -n "${INSECURE:-}" ] || echo ' - or, behind TLS inspection, INSECURE=1')
said() { local w=${why#curl: }; printf '%s' "${w%%$'\n'*}"; }   # its first line
pypi=${PIP_INDEX_URL:-https://pypi.org/simple}
why=$(curl -sSfI "${curl_k[@]}" --max-time 20 "${pypi%/}/pip/" 2>&1 >/dev/null) \
  || stop "cannot reach ${pypi%/}/pip/ ($(said)): set PIP_INDEX_URL to your PyPI mirror (chapter
    2, step 7)$tls_hint"
ok "PyPI: $pypi"
# pymssql's SQL Server client, FreeTDS, built into it from its source (execution-environment.yml,
  prepend_builder): the
# file the recipe names, checked against the SHA-256 it gives - downloaded here when it is not in
  the folder yet
if grep -q 'src: freetds.tar.gz' "$yml"; then
  ft_url=$(grep -o 'https://www\.freetds\.org/files/stable/freetds-[0-9.]*\.tar\.gz' "$yml" | head
    -1)
  ft_sum=$(grep -o '"[0-9a-f]\{64\}  /tmp/freetds.tar.gz"' "$yml" | head -1 | cut -c2-65)
  ft=$folder/freetds.tar.gz
  if [ ! -s "$ft" ]; then
    why=$(curl -sSfL "${curl_k[@]}" --max-time 300 -o "$ft.part" "$ft_url" 2>&1) && mv "$ft.part"
      "$ft" \
      || { rm -f "$ft.part"; stop "cannot download $ft_url ($(said)): download it on another
        machine and put it here as $ft (chapter 2, step 7)$tls_hint"; }
  fi
  echo "$ft_sum  $ft" | sha256sum -c --status - \
    || stop "$ft is not ${ft_url##*/} (its SHA-256 differs): rm $ft and run this again, or put
      that file there (chapter 2, step 7)"
```

build-ee.sh (continued)

```
  ok "FreeTDS for pymssql: $ft (${ft_url##*/})"
fi
[ -z "$hub" ] || why=$(curl -sSI "${curl_k[@]}" --max-time 20 "$hub" 2>&1 >/dev/null) \
  || stop "cannot reach your hub at $hub ($(said)) (chapter 2, step 4)$tls_hint"
[ -z "$hub" ] || ok "the hub answers"
if [ -n "${INSECURE:-}" ]; then
  note "INSECURE=1: no certificate is checked - pip (trusted: $PIP_TRUSTED_HOST), ansible-galaxy
    (--ignore-certs), podman (--tls-verify=false)"
elif [ -n "${PIP_TRUSTED_HOST:-}" ]; then
  note "pip does not check the certificate of: $PIP_TRUSTED_HOST"
fi

if [ -z "${TOKEN:-}" ]; then
  read -rsp "   Your automation hub token (chapter 2, step 5; nothing shows): " TOKEN || TOKEN=;
    echo
fi
[ -n "${TOKEN:-}" ] || stop "no hub token (chapter 2, step 5)"
ok "the hub token (${#TOKEN} characters)"

free=$(df -Pk "$(podman info --format '{{.Store.GraphRoot}}' 2>/dev/null || echo "$HOME")" | awk
  'NR == 2 {print int($4 / 1048576)}')
if [ "${free:-0}" -lt 15 ]; then note "only ${free:-?} GB free for podman's images: the build
  needs about 15"; else ok "${free} GB free"; fi

step "Build $image (ansible-builder, then podman: 10 to 25 minutes)"
args=(--build-arg "ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=$TOKEN"
      --build-arg "ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN=$TOKEN"
      --build-arg "ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN=$TOKEN")
[ -z "${PIP_INDEX_URL:-}" ] || args+=(--build-arg "PIP_INDEX_URL=$PIP_INDEX_URL")
[ -z "${PIP_TRUSTED_HOST:-}" ] || args+=(--build-arg "PIP_TRUSTED_HOST=$PIP_TRUSTED_HOST")
[ -z "${INSECURE:-}" ] || args+=(--build-arg ANSIBLE_GALAXY_CLI_COLLECTION_OPTS=--ignore-certs
                                 --build-arg ANSIBLE_GALAXY_CLI_ROLE_OPTS=--ignore-certs)
(cd "$folder" && ansible-builder create -f execution-environment.yml -c context) || stop
  "ansible-builder create failed: the error above (chapter 8)"
(cd "$folder" && podman build --network=host "${tls[@]}" "${args[@]}" -f context/Containerfile -t
  "$image" context) \
  || stop "the build failed: the error above (chapter 8, When something goes wrong; 'Could not
    satisfy': bash check-collections.sh $folder lists every collection your hub lacks)"
ok "built $image"

step "ee-check"
podman run --rm "$image" ee-check || stop "something is missing in $image: the lines with 'no'
  above (chapter 8)"

if [ -n "${PUSH:-}" ]; then
  step "Push to $PUSH (chapter 6, step 1)"
  podman login --get-login "$PUSH" >/dev/null 2>&1 || podman login "${tls[@]}" "$PUSH"
  podman tag "$image" "$PUSH/$name:$tag"
```

build-ee.sh (continued)

```
  podman push "${tls[@]}" "$PUSH/$name:$tag" || stop "the push failed: the error above (chapter 6,
    step 1)"
  ok "pushed $PUSH/$name:$tag"
  where="$PUSH/$name:$tag"; pull="Only pull the image if not present before running"
elif [ -n "${SAVE:-}" ]; then
  step "Save as a file (chapter 6, step 1, without a registry)"
  podman save -o "$name-$tag.tar" "$image"
  ok "saved $name-$tag.tar: on each execution node, as the user AAP runs jobs as: podman load -i
    $name-$tag.tar"
  where=$image; pull="Never pull container before running"
else
  where=$image; pull="(push it or save it first: PUSH=your-hub or SAVE=1, chapter 6, step 1)"
fi

step "Done. In AAP: Automation Execution > Infrastructure > Execution Environments > Create
  (chapter 6, step 2)"
echo "   Name:  $aap"
echo "   Image: $where"
echo "   Pull:  $pull"
echo "   An EE already in AAP: give it the new Image - the job AAP - set execution environment,
  aap_ee_name: \"$aap\""
echo "   and aap_ee_image: $where (Check first) - and every job template on it runs the new image
  (chapter 6, step 3)"
```

check-collections.sh

```
#!/usr/bin/env bash
# Does your automation hub have every collection an EE asks for? The build stops at the first one
  it cannot find;
# this checks each collection of the folder's requirements.yml at once, as the build does: one the
  base EE already
# has, in a version that fits, is kept (the hub is not asked); the others are asked of the hub -
  with the base EE's
# own ansible-galaxy and the folder's ansible.cfg - and the missing ones listed. Run it in the
  folder that holds
# windows/, network/ and all-in-one/:
#   bash check-collections.sh all-in-one
#   INSECURE=1 bash check-collections.sh all-in-one           # behind TLS inspection: no
  certificate checked
#   FETCH=1 INSECURE=1 bash check-collections.sh all-in-one   # and the missing ones downloaded
  from Galaxy (no
#       login), the newest version that fits, into collections-to-upload/ - for AAP: Automation
  Content > Namespaces >
#       the collection's namespace > Upload collection, then Collection Approvals > Approve
  (docs/EXECUTION_ENVIRONMENT.md,
#       chapter 2, step 4)
#   bash check-collections.sh all-in-one --list              # only the list it asks for
  (name:version)
# The hub token: it asks for it (nothing shows), or takes TOKEN from the environment. It is never
  written to a file
# or shown.
set -euo pipefail
cd "$(dirname "$0")"

folder=${1:-}
case "$folder" in
  windows|network|all-in-one) ;;
  *) echo "usage: bash check-collections.sh windows|network|all-in-one [--list]" >&2; exit 2 ;;
esac
# name:version of each collection in requirements.yml ("*" when it has no version: line)
reqs=$(awk '/^  - name:/ {if (n) print n ":" v; n = $3; v = "*"}
            /^    version:/ {v = $2; gsub(/"/, "", v)}
            END {if (n) print n ":" v}' "$folder/requirements.yml")
if [ "${2:-}" = --list ]; then echo "$reqs"; exit 0; fi

cfg=$folder/ansible.cfg
[ -f "$cfg" ] || { echo "no $cfg: cp $folder/ansible.cfg.example $cfg and put your hub's address
  in it (chapter 2, step 4)" >&2; exit 1; }
base=$(awk '/base_image:/ {b = 1} b && /name:/ {print $2; exit}'
  "$folder/execution-environment.yml")
podman image exists "$base" || { echo "the base image $base is not on this machine (chapter 2,
  step 3)" >&2; exit 1; }
if [ -z "${TOKEN:-}" ]; then
  read -rsp "Your automation hub token (chapter 2, step 5; nothing shows): " TOKEN || TOKEN=; echo
```

check-collections.sh (continued)

```
fi
[ -n "${TOKEN:-}" ] || { echo "no hub token (chapter 2, step 5)" >&2; exit 1; }
export TOKEN

work=$(mktemp -d); trap 'rm -rf "$work"' EXIT
cp "$cfg" "$work/ansible.cfg"; printf '%s\n' "$reqs" > "$work/reqs.txt"
out=$work
if [ -n "${FETCH:-}" ]; then out=$PWD/collections-to-upload; mkdir -p "$out"; fi

# in the base EE, as root of the container (= you: your files, readable by you only, are readable
  there)
inner=$(cat <<'INNER'
export ANSIBLE_CONFIG=/ee/ansible.cfg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=$TOKEN \
       ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN=$TOKEN ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN=$TOKEN
c=(); [ -z "${INSECURE:-}" ] || c=(--ignore-certs)
core=$(ansible --version | sed -n '1s/.*core \([0-9.]*\).*/\1/p')
py=$(head -1 "$(command -v ansible)" | sed -e 's/^#! *//' -e 's/ .*//')
fits() {  # a downloaded collection's requires_ansible, and whether this ansible-core is in it
  "$py" - "$1" "$core" <<'PY'
import sys, tarfile, yaml
from packaging.specifiers import SpecifierSet
with tarfile.open(sys.argv[1]) as t:
    try:
        ra = (yaml.safe_load(t.extractfile("meta/runtime.yml")) or {}).get("requires_ansible") or
          ""
    except KeyError:
        ra = ""
ok = not ra or SpecifierSet(ra).contains(sys.argv[2])
print("needs ansible-core %s%s" % (ra or "(any)", "" if ok else " - NOT " + sys.argv[2] + ": find
  an older one"))
PY
}
got() { sed -n "s/^Collection '\(.*\)' was downloaded successfully.*/\1/p" <<< "$1" | head -1; }
# the base EE's collections, where the build installs: one that fits is kept, the hub is not asked
  for it
ansible-galaxy collection list --format json -p /usr/share/ansible/collections 2>/dev/null >
  /tmp/have.json || echo '{}' > /tmp/have.json
in_base() {  # the base EE's version of collection $1 when it fits $2 ("*" = any version)
  "$py" - "$1" "$2" <<'PY'
import json, re, sys
from packaging.specifiers import SpecifierSet
from packaging.version import Version
name, want = sys.argv[1], sys.argv[2]
for path, cols in json.load(open("/tmp/have.json")).items():
    v = (cols.get(name) or {}).get("version") if
      path.rstrip("/").endswith("/usr/share/ansible/collections/ansible_collections") else None
    if v:
        spec = ",".join(s if re.match(r"[<>=!~]", s.strip()) else "==" + s.strip() for s in
```

check-collections.sh (continued)

```
          want.split(","))
        if want == "*" or v == "*" or SpecifierSet(spec).contains(Version(v)):
            print(v)
PY
}
n=0; missing=0
echo "== The base EE, then your hub (ansible.cfg), for ansible-core $core:"
while IFS= read -r r; do
  [ -n "$r" ] || continue
  n=$((n + 1)); name=${r%%:*}; want=${r#*:}
  if v=$(in_base "$name" "$want" 2>/dev/null) && [ -n "$v" ]; then
    printf '   ok       %-28s %s (the base EE has it)\n' "$name" "$v"
    continue
  fi
  if log=$(ansible-galaxy collection download --no-deps "${c[@]}" -p /tmp/hub "$r" 2>&1); then
    printf '   ok       %-28s %s\n' "$name" "$(got "$log" | sed 's/.*://')"
    continue
  fi
  if ! grep -q 'Could not satisfy' <<< "$log"; then
    echo "   STOP     $name: $(grep -m1 '^ERROR!' <<< "$log" | sed -e 's/^ERROR! //' -e 's/\.
      <urlopen.*//' | cut -c1-220)"
    echo "   the hub did not answer as it should: its address in ansible.cfg (chapter 2, step 4),
      the token (step 5)$([ -n "${INSECURE:-}" ] || echo ', or INSECURE=1 behind TLS inspection')"
    exit 3
  fi
  missing=$((missing + 1))
  printf '   MISSING  %-28s %s\n' "$name" "$([ "$want" = '*' ] && echo 'not on your hub' || echo
    "no version $want on your hub")"
  [ -n "${FETCH:-}" ] || continue
  if glog=$(ansible-galaxy collection download --no-deps "${c[@]}" -s https://galaxy.ansible.com/
    -p /out "$r" 2>&1); then
    f=$(got "$glog"); tgz=/out/${name/./-}-${f#*:}.tar.gz
    echo "            from Galaxy: collections-to-upload/${tgz#/out/} ($(fits "$tgz"))"
  elif grep -q 'Could not satisfy' <<< "$glog"; then
    echo "            not on Galaxy either$([[ $name != redhat.* ]] || echo " - a Red Hat
      collection: your hub's rh-certified repository syncs it")"
    echo "            or leave it out: its line out of $FOLDER/requirements.yml and
      $FOLDER/check-list.txt (chapter 2, step 4)"
  else
    echo "            Galaxy did not answer: $(grep -m1 '^ERROR!' <<< "$glog" | sed 's/^ERROR! //'
      | cut -c1-160)"
  fi
done < /ee/reqs.txt
rm -f /out/requirements.yml
if [ "$missing" = 0 ]; then echo "== all $n in the base EE or on your hub: build"; exit 0; fi
echo "== $missing of $n not on your hub$([ -n "${FETCH:-}" ] || echo ' - FETCH=1 downloads them
  from Galaxy')."
echo "   Upload each into AAP: Automation Content > Namespaces > the namespace its name begins
```

check-collections.sh (continued)

```
  with (Create namespace"
echo "   if it is not there) > Upload collection; then Collection Approvals > Approve. Then run
  this again."
exit 1
INNER
)
podman run --rm --user 0 --network=host --entrypoint bash -e TOKEN -e INSECURE -e FETCH -e
  "FOLDER=$folder" \
  -v "$work:/ee:Z" -v "$out:/out:Z" "$base" -c "$inner" 2> >(grep -v 'Failed to mount
    subscriptions' >&2)
```